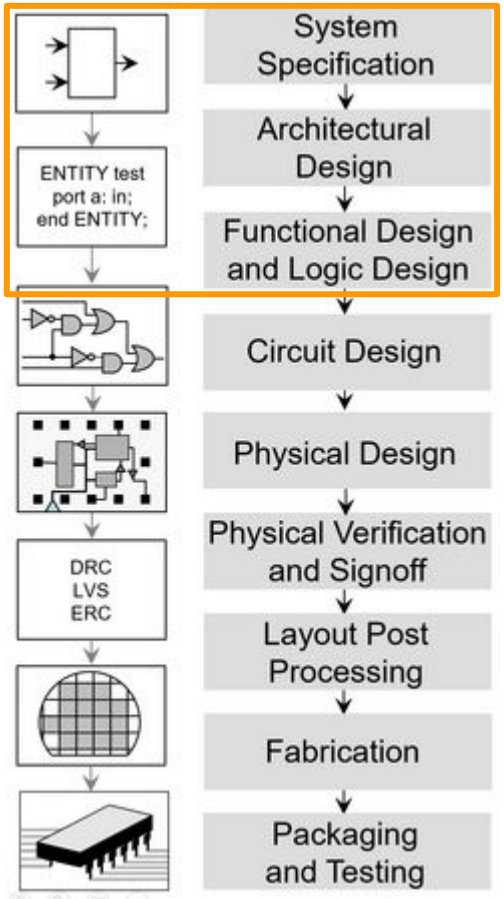
Why Verilog?

And more questions

Because we want to make chips

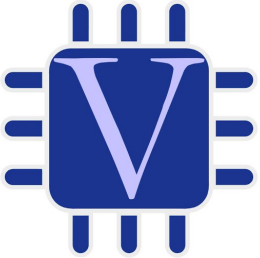
- You can't edit silicon like you can with code – errors at a later stage are significantly harder to fix
- Chip design is broken into stages:
  - * **Specification** → define what ("4-bit counter, sync reset, active-high enable")
  - * **High-Level Design** → define the pieces (ALU, decoder, registers) and their connections
  - * **Micro/Low-Level Design** → define how each piece works (FSMs, muxes, counters)
  - * **RTL Coding** → translate into Verilog/VHDL
  - * **Simulation → verify correctness (~60-70% of project time)**
  - **Synthesis** → RTL → gates
  - **Place & Route** → gates → physical layout → GDS file
  - **Post-Silicon Validation** → test the real chip
- *: Parts that we will touch upon
- [source]



What are Hardware Description Languages

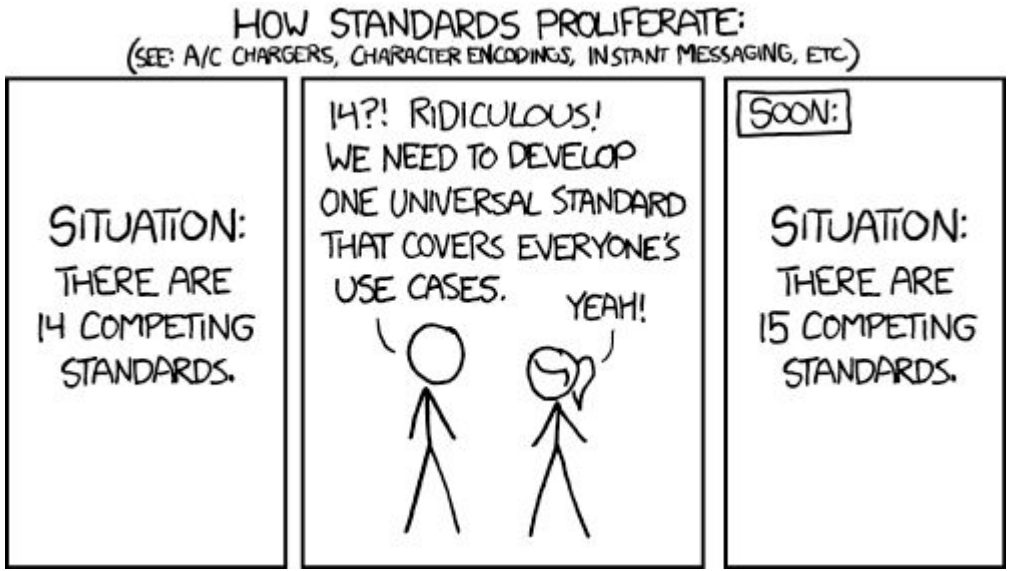
A hardware description language (HDL) is a specialized computer language used to describe the structure and behavior of electronic circuits, usually to design ASICs and to program FPGAs.

- Key difference from software languages: statements run concurrently by default (like real gates do), not line-by-line
- The same code can be:
  - **Simulated** → check correctness before anything is built
  - Synthesized → converted into an actual gate-level circuit
- Examples: Verilog, VHDL, SystemVerilog





Why another language?

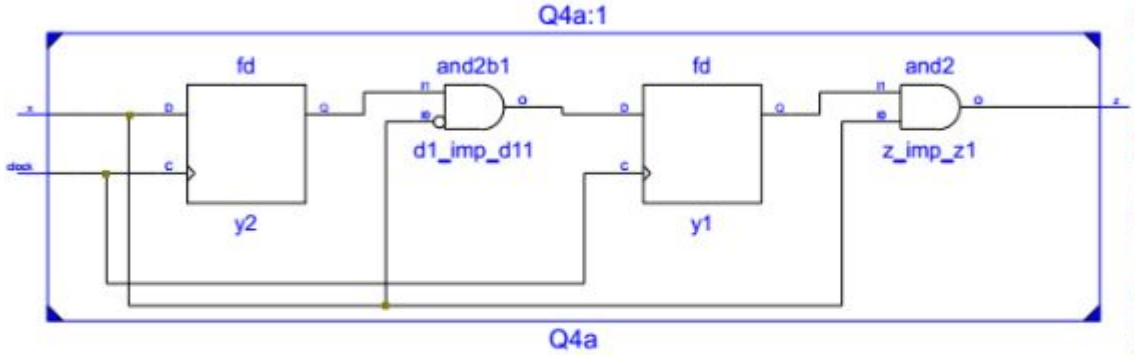


https://xkcd.com/927/

Why use Hardware Description Languages?

Relevant SO post: https://electronics.stackexchange.com/q/601664

- Because circuits are inherently **concurrent (multiple components functioning simultaneously** – hard to implement in a lang. built for the single-threaded CPU model
- Because it abstracts away details we don't care about
- Because it makes simulation easier to approach




```
module Q4a(
 input x,
 input clock,
 output reg z //Just declare as reg here
);

reg y1,y2;
reg d1,d2;

// Use automatic sensitivity list
always @* begin
  d1=(~x)&y2;
  d2=x;
  z=x&y1;
end

//Filp-flops use `posedge` to make edge sensitive
always @(posedge clock) begin
  y1<=d1;
  y2<=d2;
end

endmodule
```

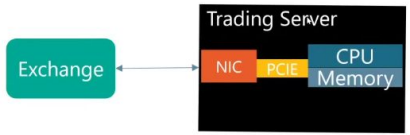
Why not more software?

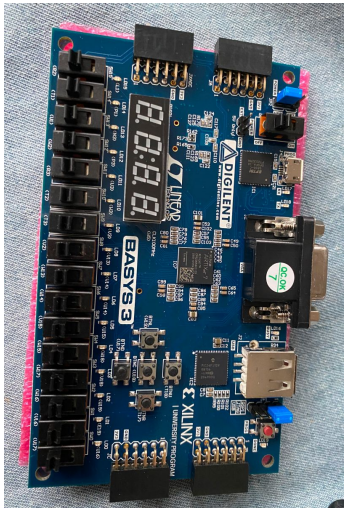
Because it's too slow for high-frequency trading

- CPUs are **general purpose** and constrained to repeat the fetch -> decode -> execute loop
- OS network stack, device drivers all add **unavoidable (and unpredictable) delay**
- An **FPGA / ASIC** allows you to hardwire computation for ultra low latency, reliable execution
- Eg: Used in high-frequency regime trading for **nanosecond** scale processing times

optiver

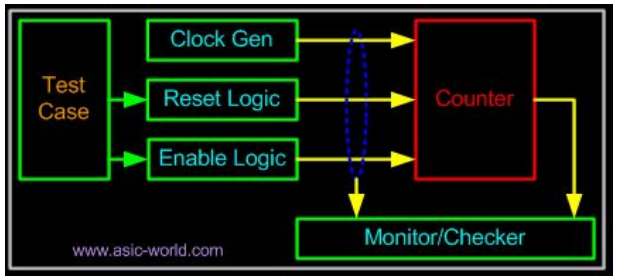
LIMITATIONS TO SOFTWARE BASED TRADING





Again, why Verilog?

- **Easy to write** — describe complex behavior (registers, FSMs, arithmetic) in a few lines, instead of hand-drawing gates and wires one by one
- **Easy to test** — write testbenches to automatically drives inputs and checks outputs
- **Easy to see** — dump every signal to a waveform and watch your circuit's behavior visually
- **Synthesizable** (not covered) — converting the RTL to a real gate-level circuit on an FPGA/ASIC




```
1  module addbit (
2  a      , // first input
3  b      , // Second input
4  ci     , // Carry input
5  sum    , // sum output
6  co       // carry output
7  );
8  //Input declaration
9  input a;
10 input b;
11 input ci;
12 //Ouput declaration
13 output sum;
14 output co;
15 //Port Data types
16 wire   a;
17 wire   b;
18 wire   ci;
19 wire   sum;
20 wire   co;
21 //Code starts here
22 assign {co,sum} = a + b + ci;
23
24 endmodule // End of Module addbit
```

Some definitions to help

For useful links and resources check the repo.

- **Testbench** — code that isn't part of the real design; it just drives inputs into your circuit and checks outputs
- **Simulation** — running your circuit design virtually, before it's real, to check correctness
- **Waveform** — a timeline view of every signal in your circuit, used to visually debug behavior
- **Clock (clk)** — a repeating signal that ticks at a fixed rate
- **Module** — the basic building block in Verilog; a reusable, self-contained unit of hardware
- **Combinational logic** — output depends only on current inputs, no memory (e.g. an adder, a mux)
- **Sequential logic** — output depends on current inputs and past state (e.g. a counter, a register) — needs a clock
- **Register/Flip-flop** — a 1-bit memory element that holds its value until the next clock edge
- **Reset (rst)** — a signal that forces the circuit into a known starting state, ignoring everything else
- **Wire** – a signal that carries a value from one point to another; it doesn't store anything

Not relevant to verilog but mentioned previously:

- **RTL (Register Transfer Level)** — describing a circuit as data moving between registers through logic
- **FPGA (Field-Programmable Gate Array)** — a reprogrammable chip that can be configured based on HDL code
- **ASIC (Application-Specific Integrated Circuit)** — a chip custom-fabricated to do exactly one thing

Verilog In One Day

https://www.asic-world.com/verilog/veritut.html

Basic Verilog Syntax

- A **module** is the fundamental block of digital computation in Verilog

```
module priority_decoder_4to2 (
    input  wire [3:0] in,         // 4 input request lines
    output reg  [1:0] code,       // encoded index of highest-priority input
    output reg        valid       // 1 if any input is asserted
);…

endmodule
```

  - `input`/`output` are keywords to define direction of port (`inout` port can be used for both)
  - `// …` and `/* */` are for comments
  - `wire/reg` define data types and `[N:0]` defines size; we will get to them later
  - Each module definition must define ports (inputs & outputs) in parentheses
  - Module definition starts with `module` and ends with `endmodule`
  - **Note:** Curly braces are replaced with explicit `begin`, `end...` -style statements in Verilog
  - Statements are semicolon(`;`) terminated

Instantiating a Module

- define variables for **input/output**

  for eg: `req_lines, priority_code, req_valid`

- instantiate it with a **unique instance name**

  for eg: `pdec_inst` is the unique identifier for this instance of `priority_decoder_4to2`

- wire in the inputs/outputs using **port names**

  for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg  [3:0] req_lines;     // input - reg
wire [1:0] priority_code; // output - wire
wire       req_valid;     // output - wire

priority_decoder_4to2 pdec_inst (
    .in    (req_lines),
    .code  (priority_code),
    .valid (req_valid)
);
```

Data Types

Verilog supports two logical data types:

- `wire` – carries a value **continuously** from whatever drives it; has no memory of its own
  - `assign` is used for **continuous assignment** to a wire. It cannot be used for register data.
  - Eg: `assign q_bar = ~q_reg;` here `q_bar` is a wire which is being “assigned” a value

- `reg` — a variable that **holds its value** until explicitly assigned inside a procedural block
  - Recall what a physical register / flip-flop means from your theory course
  - **Note:** The definition is **logical**. It has the option but not the guarantee to synthesize to a **physical register**
  - Can only be assigned inside of `always @(...)` blocks

Two ways to do assignment

- `always` blocks define code that is always active
- we can write **combinational** or **sequential** logic here

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (=) is used
- Eg (explanation in lab)

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (<=) is used
- Eg (explanation in lab)

```
always @(posedge clk) begin
    if (!rst_n)
        q_reg <= 1'b0;
    else
        q_reg <= d;
end
```

Back to Basics

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **`integer`** – plain whole number variable, mostly used to count things (like loop iterations)

  for eg: `integer i; for (i = 0; i < 16; i = i + 1)`.

- **`parameter`** — a named constant you can **reconfigure** from outside the module

  for eg: `parameter WIDTH = 8` – can write variable-width modules using this

One is^{n}t gud genug

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name` – indices are **inclusive**

  for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<bit-width>'<base><value>`

  for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment**:

  `data = 4'b0110` (full assignment) `data[1:0] = 2'b10` (multi-bit assignment)

  `data[3:0] = {2'b01, 2'b10}` (using concat operator)

  > this will assign the bits 3, 2, 1, 0 (MSB on the left to LSB on the right) to the values 0, 1, 1, 0 resp.

- **Reading:** Same syntax as assignment

  for eg: `data[1:0]==2'b10` will work

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Testing Verilog

```
`timescale 1ns/1ns

module tb;
    reg a, b, cin;
    wire cout, sum;

    adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        $monitor("At time %0t: a = %b, b = %b, \
        cin = %b -> cout = %b, sum = %b",
                 $time, a, b, cin, cout, sum);
    end

    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);
```

```
    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);

        a = 0; b = 0; cin = 0; #10;
        a = 0; b = 0; cin = 1; #10;
        a = 0; b = 1; cin = 0; #10;
        a = 0; b = 1; cin = 1; #10;
        a = 1; b = 0; cin = 0; #10;
        a = 1; b = 0; cin = 1; #10;
        a = 1; b = 1; cin = 0; #10;
        a = 1; b = 1; cin = 1; #10;

        $display("Done!")
        $finish;
    end
endmodule
```

Testing Verilog

```
`timescale 1ns/1ns

module tb;
    reg a, b, cin;
    wire cout, sum;

    adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        $monitor("At time %0t: a = %b, b = %b, \
        cin = %b -> cout = %b, sum = %b",
                 $time, a, b, cin, cout, sum);
    end

    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);
```

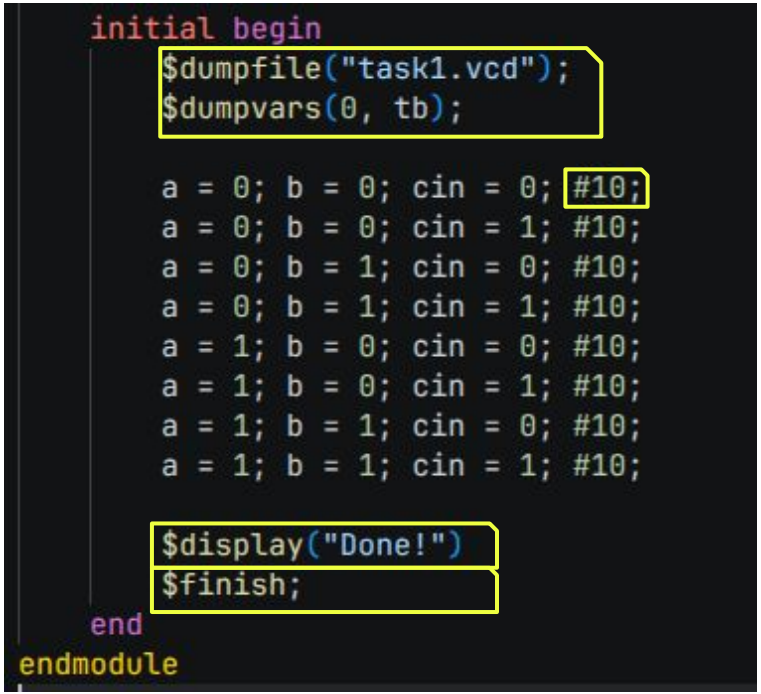
> ***A testbench for task1***

- `` `timescale <unit>/<precision> `` for simulation
- N time units are represented as `#N` .

- use **`reg`** to store **inputs**, and **`wire`** to take **outputs**
- **instantiate** the module as `uut` and drive inputs/outputs for testing

- **`initial`** block will **run once** at T=0; used for setup code
- **`$monitor`** prints a message anytime **one of the inputs changes**

Testing Verilog



```
    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);

        a = 0; b = 0; cin = 0; #10;
        a = 0; b = 0; cin = 1; #10;
        a = 0; b = 1; cin = 0; #10;
        a = 0; b = 1; cin = 1; #10;
        a = 1; b = 0; cin = 0; #10;
        a = 1; b = 0; cin = 1; #10;
        a = 1; b = 1; cin = 0; #10;
        a = 1; b = 1; cin = 1; #10;

        $display("Done!")
        $finish;
    end
endmodule
```

- **`$dumpfile`** and **`$dumpvars`** tell the simulator where to store waveform data and which variables to dump
- define delays in terms of the unit **(1ns here)** using **`#10`** etc. for eg: each consecutive line runs after 10ns
- **`$display`** prints a message once
- **`$finish`** tells the simulator to end simulating

Testing Verilog

Output from vvp command:

```
VCD info: dumpfile task1.vcd opened for output.
At time 0: a = 0, b = 0,         cin = 0 -> cout = 0, sum = 0
At time 10: a = 0, b = 0,         cin = 1 -> cout = 0, sum = 1
At time 20: a = 0, b = 1,         cin = 0 -> cout = 0, sum = 1
At time 30: a = 0, b = 1,         cin = 1 -> cout = 1, sum = 0
At time 40: a = 1, b = 0,         cin = 0 -> cout = 0, sum = 1
At time 50: a = 1, b = 0,         cin = 1 -> cout = 1, sum = 0
At time 60: a = 1, b = 1,         cin = 0 -> cout = 1, sum = 0
At time 70: a = 1, b = 1,         cin = 1 -> cout = 1, sum = 1
Done!
task1/tb_task1.v:35: $finish called at 80 (1ns)
```

Compiling Verilog

```
$ iverilog adder.v tb_task1.v -o task1.vvp
$ vvp task1.vvp
```

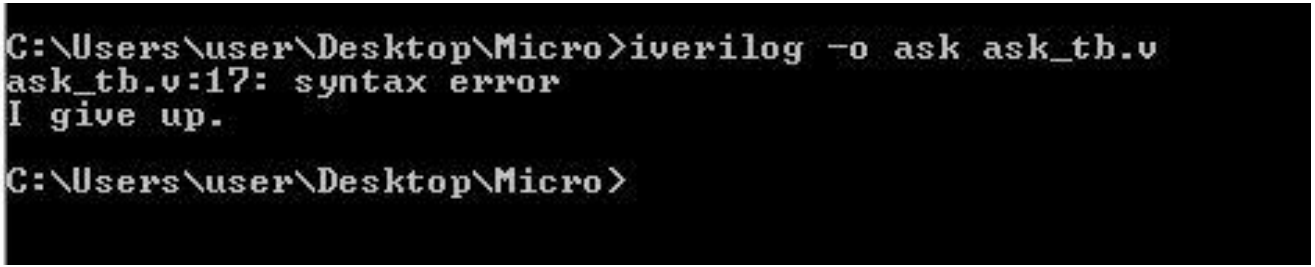
1. **Compilation**: compile **all verilog files** using **`iverilog`** (Icarus Verilog) into a **.vvp** output

2. **Simulation:** run the simulation using **`vvp`**; this will execute the blocks in the **testbench** and display+dump suitable output

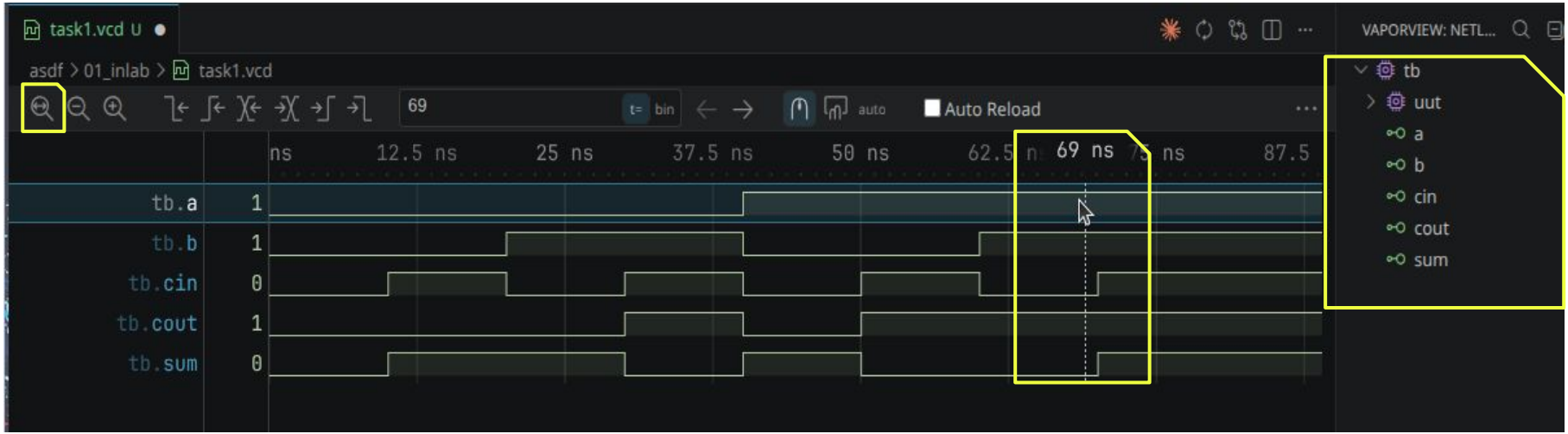
3. **Viewing waveform**: open the (optionally) generated **`.vcd`** file in **Vaporview in VS Code**

I give up

Syntax errors are common. Don’t give up.

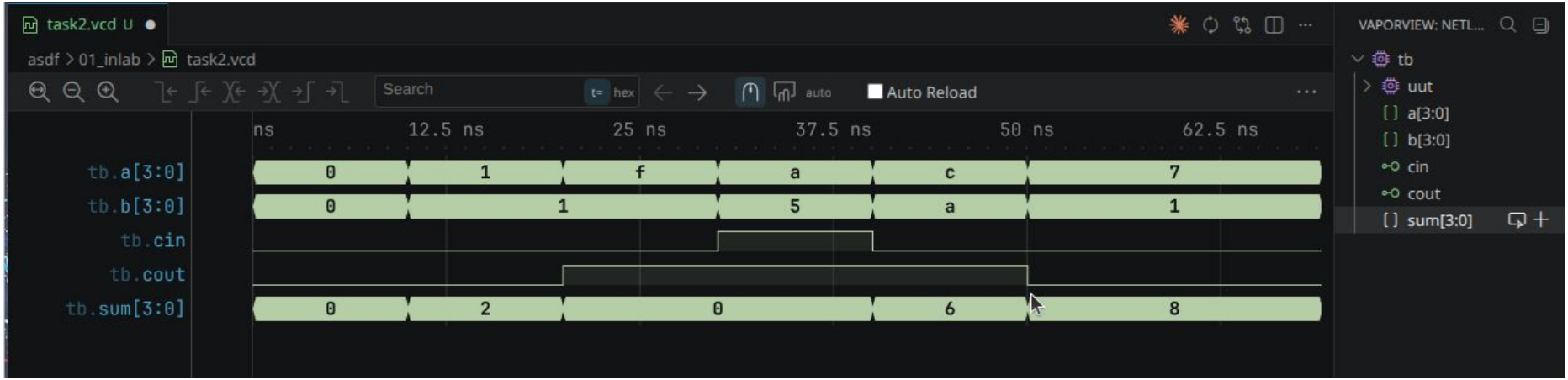


Seeing Verilog



- `$dumpfile/$dumpvars` are used to dump signal values over some simulation time to **.vcd** file
- you can view the waveforms evolving over time in a viewer such as GTKWave, or **Vaporview on VS Code**
- select the signals for display in the **Netlist View**, in the **Vaporview tab in the sidebar**
- click **Zoom to Fit**; then click anywhere (eg. 69ns here) to see values at that point

Seeing Verilog



- vector signals can also be inspected

It’s graded lab time



Submit onto:
Week 1 Inlab Assessment

run ./build_archive.sh
Type your roll no.
Submit the tar file generated
File name: <roll-no>.tar.gz

Verilog (week 2): Brand New Day

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab



Back to Basics: Module Definition

- Module definition syntax

```
module priority_decoder_4to2 (
    input  wire [3:0] in,         // 4 input request lines
    output reg  [1:0] code,       // encoded index of highest-priority input
    output reg        valid       // 1 if any input is asserted
);…

endmodule
```

  - `input`/`output` are keywords to define direction of port (`inout` port can be used for both)
  - `// …` and `/* */` are for comments
  - `wire`/`reg` define data types and `[N:0]` defines size; we will get to them later
  - Each module definition must define ports (inputs & outputs) in parentheses
  - Module definition ends with `endmodule`
  - **Note:** Curly braces are replaced with explicit `begin`, `end...` -style statements in Verilog

Back to Basics: Instantiation

- define variables for **input/output**

  for eg: `req_lines, priority_code, req_valid`

- instantiate it with a **unique instance name**

  for eg: `pdec_inst` is the unique identifier for this instance of `priority_decoder_4to2`

- wire in the inputs/outputs using **port names**

  for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg  [3:0] req_lines;     // input - reg
wire [1:0] priority_code; // output - wire
wire       req_valid;     // output - wire

priority_decoder_4to2 pdec_inst (
    .in    (req_lines),
    .code  (priority_code),
    .valid (req_valid)
);
```

Back to Basics: Can you C it?

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

Back to basics: One is^{n}t gud genug

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

  for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

  for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment**:

  `data = 4'b0110` (full assignment) `data[1:0] = 2'b10` (multi-bit assignment)

  `data[3:0] = {2'b01, 2'b10}` (using concat operator)

  > this will assign the bits 3, 2, 1, 0 (left to right) to the values 0, 1, 1, 0.

- **Reading:** Same syntax as assignment

  for eg: `data[high:low]` will read bits `low,low+1,...high-1,high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Back to Basics: More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **`integer`** – plain whole number variable, mostly used to count things (like loop iterations)

  for eg: `integer i; for (i = 0; i < 16; i = i + 1)`.

- **`parameter`** — a named constant you can **reconfigure** from outside the module

  for eg: `parameter WIDTH = 8` – can write variable-width modules using this

- `real` — a decimal/floating-point number; **only used in simulation/testing**

  for eg: `real voltage = 3.3`.

- `time` — stores a **simulation/testing timestamp**, usually paired with `$time`

  for eg: `time start_time; start_time = $time;`.

- `localparam` — a named constant that is **fixed inside a module** – used to store helper values

  for eg: `localparam OP_ADD = 4'b0000;`.

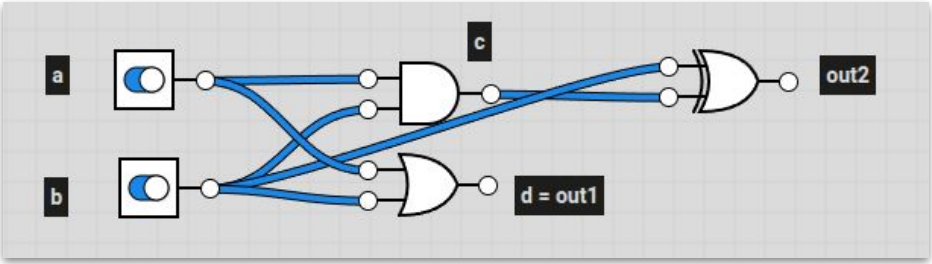
And two roads diverged...

- You will think of digital logic as either combinational or sequential
  - **Combinational** — output depends only on current inputs; no memory, no clock
  - **Sequential** — output depends on current inputs and past state; needs a clock

- Let’s see them in more detail

.. and the other, just as fair ..

- Combinational circuits are a **direct mapping** from **input signals** to **output signals**.
- Eg:



- Regardless of what the previous input was the output for a given input is the same.
- You can represent them using simple gates – AND, OR, NOT – chained together
- There is **no memory** and **no notion of a clock**
- We can say the output is “always” the same depending on the input
- Eg: <u>using assign</u>

```
assign sum = a ^ b ^ cin;
```

<u>using always @(*)</u>

```
always @(*) begin
    if (s)
        o = a;
    else
        o = b;
end
```

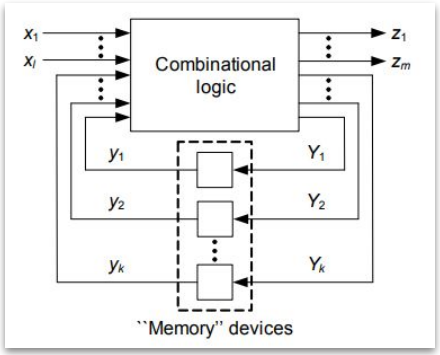
Theory Terms

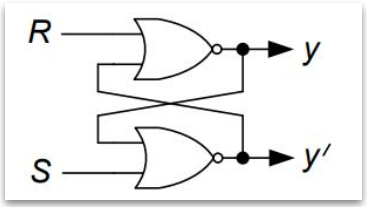
- Recall the following from CS230:
  - **Clock**: A periodic signal (typically square wave) that triggers updates in sequential circuits.
  - **Level-triggered**: A circuit that responds continuously **while a control signal is high** (or low).
  - **Edge-triggered**: A circuit that responds only at **an instant** — when the clock transitions from **0→1 (posedge)** ..... or ..... **1→0 (negedge)**

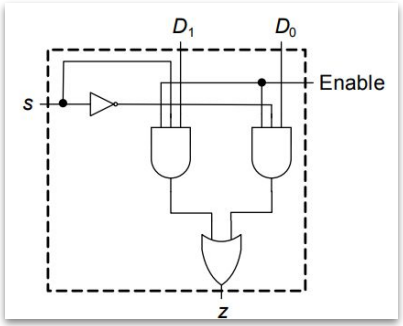
  - **D Flip-Flop (edge-triggered)**: A circuit that samples its input on a clock edge and holds that value until the next edge
  - **T Flip-Flop (edge-triggered)**: A circuit that toggles its output (flips 0↔1) when both the clock edge arrives and the toggle input is 1

.. I took the one less travelled by

- Sequential circuits are a **mapping from (input, previous state) ↦ (output, next state)**
- Eg: general sequential circuit = latches + combinational logic







- Think of registers/latches as **abstract "data stores"** and treat the seq circuit as combinational logic applied in a **clocked** (update based clock edge/level) and **deferred** (one write per clock edge) manner

- **What do we need?** (1) registers (2) clock-based triggering (3) deferring of updates

Sequential is All You Need

- We intend to cover how verilog provides these 3 abstractions:

  a. **Registers**: Defining `reg` variables for “memory”

  b. **Clock-based triggering**: Using `always` with suitable trigger list eg: `always @(posedge clk)`

  c. **Deferring of updates:** Using non-blocking assignment `<=` to defer writes until all blocks have read

Data Types Revisited

Verilog supports two data types for these purposes:

- `wire` – carries a value **continuously** from whatever drives it; has no memory of its own
  - `assign` is used for **continuous assignment** to a wire. It cannot be used for register data.
  - Eg: `assign q_bar = ~q_reg;`

- `reg` — a variable that **holds its value** until explicitly assigned inside a procedural block
  - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
  - Can only be assigned inside of `always @(...)` blocks
  - Usage: `always @(trigger-list)`
    - Eg: `always @(posedge clk)` executes on **rising edge of `clk` signal** -> **sequential**
    - Eg: `always @(*)` executes whenever **any input changes** -> **combinational**

Always there for you

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (`=`) is used
- written inside `always @(*)` blocks
- the blocking statements run in **program order**
- Eg: some random combinational logic here

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (`<=`) is used
- written inside `always @(posedge clk)` blocks (or a variant)
- Eg: naive 2-bit (modulo 4) counter

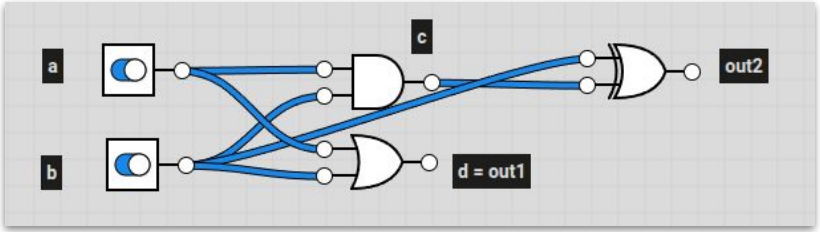
```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

Always there for you (still)

Combinational

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

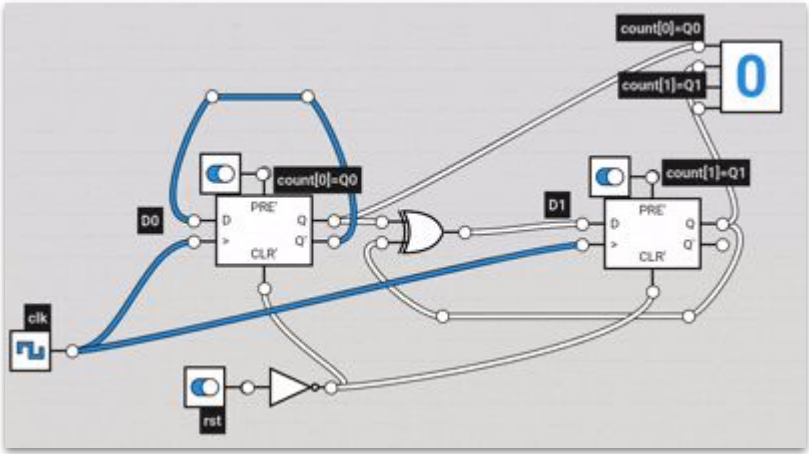
- Circuit will respect the program order for computation



Sequential

```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

- Circuit will respect the clock-based update



Quiz

- Is there something wrong with this code snippet? (Assume usual T Flip Flop semantics)

```
module tff_wire (
    input clk,
    input t,
    output wire q
);
    assign q = clk ? (t ^ q) : q;
endmodule
```

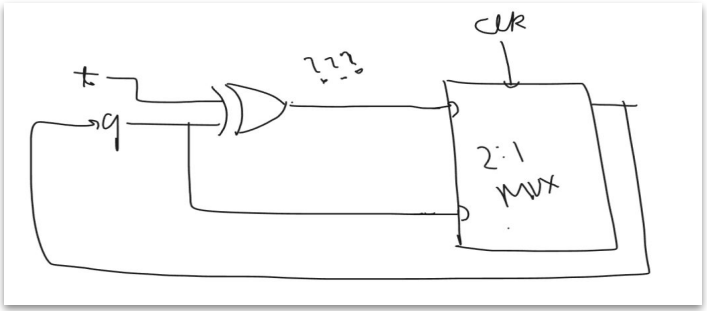
Answer: `wire` isn't enough

- Recall from CS230:

  Q_{t+1} = T ⊕ Q_t satisfies T-flip flop semantics

- But this is **level-triggered**, not **edge-triggered!**
- Plus, looping an output back in as input is wrong.
- **Wire `q` cannot settle** on a value as long as `clk` is on and `t` is set (it will keep switching between 0/1):
- This leads to **undefined state (X)** for `q`

```
module tff_wire (
    input clk,
    input t,
    output wire q
);
    assign q = clk ? (t ^ q) : q;
endmodule
```



- **We need to use reg**

- Another issue is the lack of a reset pin but let's ignore that for now

Quiz

- Let's use a `reg` now.
- Is there something wrong with this code snippet? (Assume usual T Flip Flop semantics)

```
module tff_reg (
    input clk,
    input t,
    output reg q
);
    always @(posedge clk) begin
        q = t ^ q;
    end
endmodule
```

Answer: Is it really sequential?

- This satisfies T flip flop semantics + edge-triggered correctly
- Appears that `q` will update **exactly on posedge (instantaneously)**
- **On posedge: evaluate `t^q`, set `q` to that value, repeat.**



- But **blocking assignment is wrong** here
- Doesn't guarantee deferring of updates, i.e., **one update per time step**

```
module tff_reg (
    input clk,
    input t,
    output reg q
);
    always @(posedge clk) begin
        q = t ^ q;
    end
endmodule
```

Quiz (optional)

- What's wrong with this code snippet?
  - Will `tff2.q` toggle once every 2 time periods as intended?

```
module chain_top (
    input clk,
    output q1,
    output q2
);
    // always toggles      => switches once per clock time period
    tff_reg tff1 (.clk(clk), .t(1'b1), .q(q1));
    // toggles when q1==1  => switches once per 2 clock time periods
    tff_reg tff2 (.clk(clk), .t(q1),   .q(q2));
endmodule
```

Answer: Don’t block me

- **TLDR: Use `<=` for making updates properly timed (or non-blocking) in a sequential circuit.**

- Blocking assignment (`=`) updates many variables **instantly** — this violates sequential circuit semantics which requires **all writes to be deferred to the end** and **executed as one unit**
- With a single flip-flop, this doesn't show up as an issue, as we saw in `tff_reg`
- But chain two `tff_reg`'s together (`q` output of first is fed as `t` input of the second) and now **both `always` blocks will fire on the same rising edge**
- Since `=` writes immediately, it will **not defer writes to the end**; any always block can do a write first, leading to a race condition.
- Hence, output is implementation dependent. It also depends on whether `tff1` is initialized first or `tff2` is.

Answer: Don’t block me

- Here `_nb` refers to the correct, non-blocking version;

  And `_12` and `_21` refer to order of initialization of `tff1/2` in the wrong, blocking version
- All `q1_*` toggle per 1 time period as intended (deferring of updates is only an issue with >1 interfering always blocks)
- `q2_nb` correctly toggles every 2 time periods
- `q2_12` does not toggle at all!
- `q2_21` toggles every 1 time period!



Do it once

- `<=` stands for non-blocking assignment – this allows us to defer updates to the end and execute them together

- How it works at a high level:
  - Read all old values (occurring on right hand side first
  - Then update all left hand side values **together**

- Why is it called “non-blocking”?

  All writes are **scheduled** to happen later, i.e., it **doesn’t block** any other statement trying to read the **old value**.

  A blocking assignment (`=`) will write the new value immediately and block anyone else trying to read the old value.

- **Rule of thumb:** Always use `<=` inside `always @(posedge clk)` or some variant.

- **Note: `<=` and `=` (other than `assign <wire> =` statements) can only be used in `always` or `initial` blocks.**

Example: DFF

```
module dff_with_inverter (
    input  wire clk,
    input  wire rst_n,
    input  wire d,
    output wire q,
    output wire q_bar
);

    reg q_reg;

    // ---- Sequential logic ----
    // Output depends on current input AND past state (memory)
    // Only updates on the clock's rising edge
    always @(posedge clk) begin
        if (!rst_n)
            q_reg <= 1'b0;
        else
            q_reg <= d;
    end

    // ---- Combinational logic ----
    // Output depends only on current input, no memory, no clock
    assign q     = q_reg;
    assign q_bar = ~q_reg;

endmodule
```

Example: DFF testbench

```
`timescale 1ns/1ns

module tb_dff_with_inverter;

    reg  clk, rst_n, d;
    wire q, q_bar;

    dff_with_inverter dut (
        .clk(clk), .rst_n(rst_n),
        .d(d), .q(q), .q_bar(q_bar)
    );

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dff_wave.vcd");
        $dumpvars(0, tb_dff_with_inverter);

        clk = 0; rst_n = 0; d = 0;

        #12 rst_n = 1;   // release reset

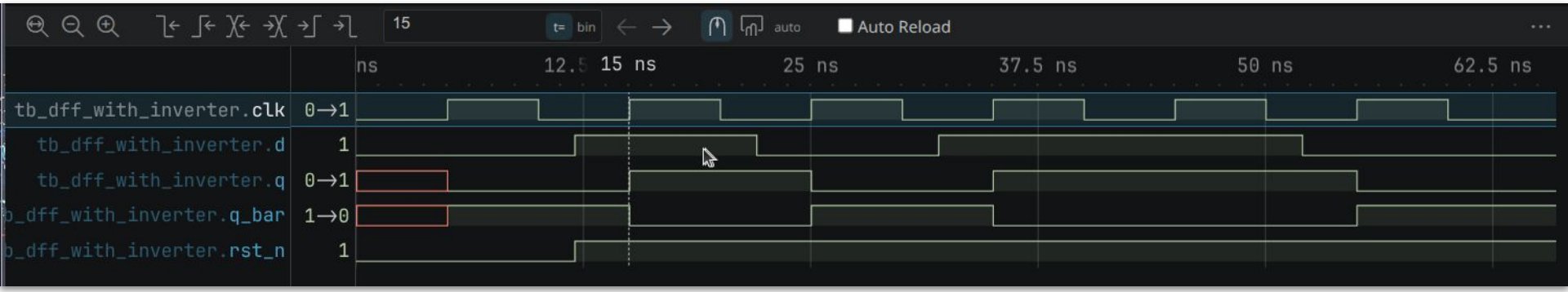
        d = 1; #10;
        d = 0; #10;
        d = 1; #10;
        d = 1; #10;
        d = 0; #10;

        $display("Test complete.");
        $finish;
    end

    initial begin
        $monitor("t=%0t rst_n=%b d=%b q=%b q_bar=%b",
                 $time, rst_n, d, q, q_bar);
    end

endmodule
```

Example: DFF Waveform



- An example testbench waveform for the D-flip flop implementation
- Note clock time period is 10ns
- `d` update is propagated to `q` only on **rising edge** when `rst_n` is set

It’s inlab time



It’s graded lab time



1. Log in to **CS231-labexam**, password will be shared
2. Extract labexam tarball and work in the extracted folder
3. Go into the same folder and run ./build-archive.sh
4. Log in to moodle and submit into the **correct activity: …**

Verilog (week 3): some assembly required

Recap slides follow

Back to Basics: Module Definition

- Module definition syntax

```
module priority_decoder_4to2 (
    input  wire [3:0] in,         // 4 input request lines
    output reg  [1:0] code,       // encoded index of highest-priority input
    output reg        valid       // 1 if any input is asserted
);…

endmodule
```

  - `input`/`output` are keywords to define direction of port (`inout` port can be used for both)
  - `// …` and `/* */` are for comments
  - `wire`/`reg` define data types and `[N:0]` defines size; we will get to them later
  - Each module definition must define ports (inputs & outputs) in parentheses
  - Module definition ends with `endmodule`
  - **Note:** Curly braces are replaced with explicit `begin`, `end...` -style statements in Verilog

Back to Basics: Instantiation

- define variables for **input/output**

  for eg: `req_lines, priority_code, req_valid`

- instantiate it with a **unique instance name**

  for eg: `pdec_inst` is the unique identifier for this instance of `priority_decoder_4to2`

- wire in the inputs/outputs using **port names**

  for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg  [3:0] req_lines;     // input - reg
wire [1:0] priority_code; // output - wire
wire       req_valid;     // output - wire

priority_decoder_4to2 pdec_inst (
    .in    (req_lines),
    .code  (priority_code),
    .valid (req_valid)
);
```

Back to Basics: Can you C it?

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

Back to basics: One is^{n}t gud genug

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

  for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

  for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment**:

  `data = 4'b0110` (full assignment) `data[1:0] = 2'b10` (multi-bit assignment)

  `data[3:0] = {2'b01, 2'b10}` (using concat operator)

  > this will assign the bits 3, 2, 1, 0 (left to right) to the values 0, 1, 1, 0.

- **Reading:** Same syntax as assignment

  for eg: `data[high:low]` will read bits `low,low+1,...high-1,high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Back to Basics: More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **`integer`** – plain whole number variable, mostly used to count things (like loop iterations)

  for eg: `integer i; for (i = 0; i < 16; i = i + 1)`.

- **`parameter`** — a named constant you can **reconfigure** from outside the module

  for eg: `parameter WIDTH = 8` – can write variable-width modules using this

- `real` — a decimal/floating-point number; **only used in simulation/testing**

  for eg: `real voltage = 3.3`.

- `time` — stores a **simulation/testing timestamp**, usually paired with `$time`

  for eg: `time start_time; start_time = $time;`.

- `localparam` — a named constant that is **fixed inside a module** – used to store helper values

  for eg: `localparam OP_ADD = 4'b0000;`.

Theory Terms

- Recall the following from CS230:
  - **Clock**: A periodic signal (typically square wave) that triggers updates in sequential circuits.
  - **Level-triggered**: A circuit that responds continuously **while a control signal is high** (or low).
  - **Edge-triggered**: A circuit that responds only at **an instant** — when the clock transitions from **0→1 (posedge)** ..... or ..... **1→0 (negedge)**

  - **D Flip-Flop (edge-triggered)**: A circuit that samples its input on a clock edge and holds that value until the next edge
  - **T Flip-Flop (edge-triggered)**: A circuit that toggles its output (flips 0↔1) when both the clock edge arrives and the toggle input is 1

Back to basics: Data Types

Verilog supports two data types for these purposes:

- `wire` – carries a value **continuously** from whatever drives it; has no memory of its own
  - `assign` is used for **continuous assignment** to a wire. It cannot be used for register data.
  - Eg: `assign q_bar = ~q_reg;`

- `reg` — a variable that **holds its value** until explicitly assigned inside a procedural block
  - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
  - Can only be assigned inside of `always @(...)` blocks
  - Usage: `always @(trigger-list)`
    - Eg: `always @(posedge clk)` executes on **rising edge of `clk` signal** -> **sequential**
    - Eg: `always @(*)` executes whenever **any input changes** -> **combinational**

Coming back to you (always)

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (`=`) is used
- written inside `always @(*)` blocks
- the blocking statements run in **program order**
- Eg: some random combinational logic here

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (`<=`) is used
- written inside `always @(posedge clk)` blocks (or a variant)
- Eg: naive 2-bit (modulo 4) counter

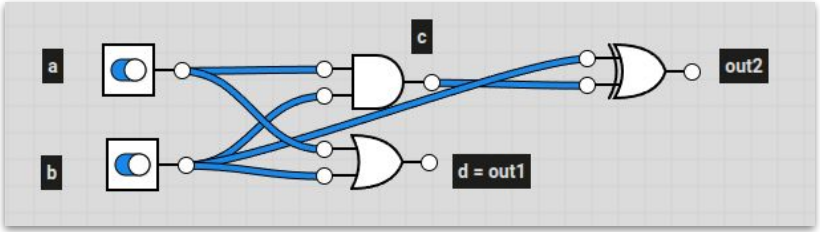
```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

Coming back to you (always)

Combinational

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

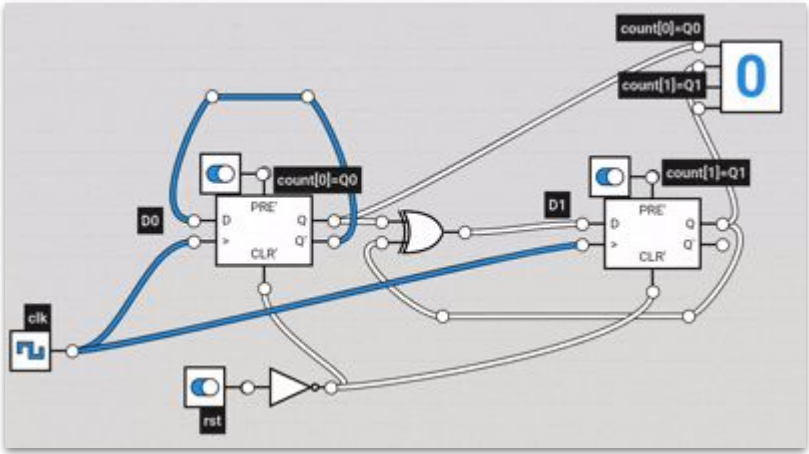
- Circuit will respect the program order for computation



Sequential

```
always @(posedge clk) begin
    if (reset)
        count <= 2'b00;
    else
        count <= count + 2'b01;
end
```

- Circuit will respect the clock-based update



How NOT to TFF it!

1. `tff_wire`

   Need **reg** not **wire**; Need **edge-triggered** not **level-triggered**

```
module tff_wire (
    input clk,
    input t,
    output wire q
);
    assign q = clk ? (t ^ q) : q;
endmodule
```

2. `tff_reg`

   Blocking assignment gives wrong assignment, need **<=**

```
module tff_reg (
    input clk,
    input t,
    output reg q
);
    always @(posedge clk) begin
        q = t ^ q;
    end
endmodule
```

Why you should not block (me)

- `<=` stands for non-blocking assignment – this allows us to defer updates to the end and execute them together

- How it works at a high level:
  - Read all old values (occurring on right hand side first
  - Then update all left hand side values **together**

- Why is it called “non-blocking”?

  All writes are **scheduled** to happen later, i.e., it **doesn’t block** any other statement trying to read the **old value**.

  A blocking assignment (`=`) will write the new value immediately and block anyone else trying to read the old value.

- **Rule of thumb:** Always use `<=` inside `always @(posedge clk)` or some variant.

- **Note: `<=` and `=` (other than `assign <wire> =` statements) can only be used in `always` or `initial` blocks.**

Example: DFF

```
module dff_with_inverter (
    input  wire clk,
    input  wire rst_n,
    input  wire d,
    output wire q,
    output wire q_bar
);

    reg q_reg;

    // ---- Sequential logic ----
    // Output depends on current input AND past state (memory)
    // Only updates on the clock's rising edge
    always @(posedge clk) begin
        if (!rst_n)
            q_reg <= 1'b0;
        else
            q_reg <= d;
    end

    // ---- Combinational logic ----
    // Output depends only on current input, no memory, no clock
    assign q     = q_reg;
    assign q_bar = ~q_reg;

endmodule
```

Example: DFF testbench

```
`timescale 1ns/1ns

module tb_dff_with_inverter;

    reg  clk, rst_n, d;
    wire q, q_bar;

    dff_with_inverter dut (
        .clk(clk), .rst_n(rst_n),
        .d(d), .q(q), .q_bar(q_bar)
    );

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dff_wave.vcd");
        $dumpvars(0, tb_dff_with_inverter);

        clk = 0; rst_n = 0; d = 0;

        #12 rst_n = 1;   // release reset

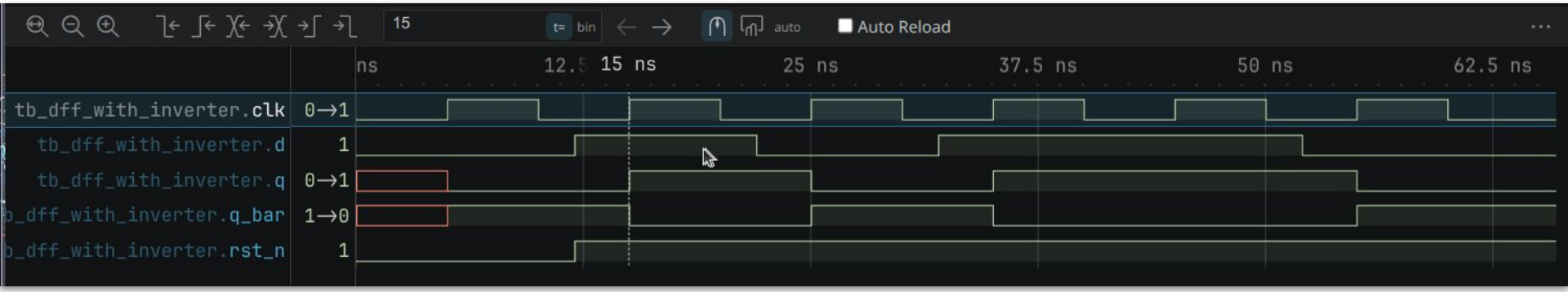
        d = 1; #10;
        d = 0; #10;
        d = 1; #10;
        d = 1; #10;
        d = 0; #10;

        $display("Test complete.");
        $finish;
    end

    initial begin
        $monitor("t=%0t rst_n=%b d=%b q=%b q_bar=%b",
                 $time, rst_n, d, q, q_bar);
    end

endmodule
```

Example: DFF Waveform



- An example testbench waveform for the D-flip flop implementation
- Note clock time period is 10ns
- `d` update is propagated to `q` only on **rising edge** when `rst_n` is set

Buffer Slide

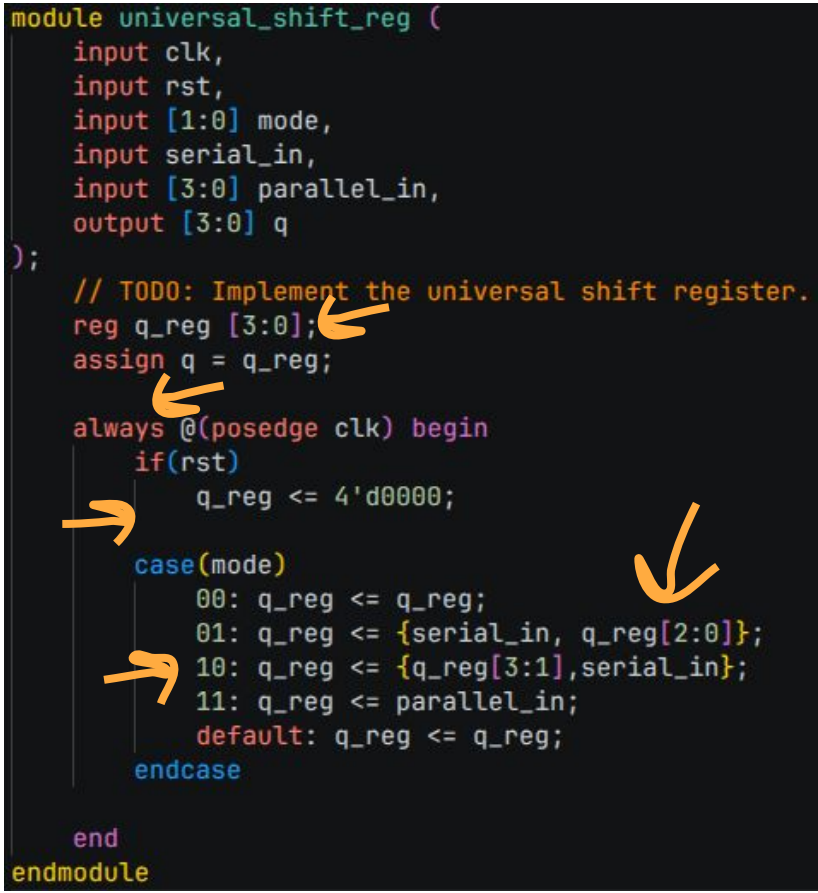
Discussing doubts/solutions attempts for last week's inlab / labexam / outlab



These should be doubts

Common problems you guys ran into in the last labexam:

1. Use `reg [3:0] q_reg` – **Single 4-bit register – Better choice**
   NOT `reg q_reg [3:0]` – an array of 1-bit regs
2. A `if` and `case` always occur inside an `always` block
3. Binary literals **need to be preceded with `2'b`** here
4. Do not miss the `else` **clause** here
5. Be careful with **bit indexing** for left and right shift
6. Don't run `initial q = 4'b0` in the module, let `rst` handle it



```
module universal_shift_reg (
    input clk,
    input rst,
    input [1:0] mode,
    input serial_in,
    input [3:0] parallel_in,
    output [3:0] q
);
    // TODO: Implement the universal shift register.
    reg q_reg [3:0];
    assign q = q_reg;

    always @(posedge clk) begin
        if(rst)
            q_reg <= 4'd0000;

        case(mode)
            00: q_reg <= q_reg;
            01: q_reg <= {serial_in, q_reg[2:0]};
            10: q_reg <= {q_reg[3:1],serial_in};
            11: q_reg <= parallel_in;
            default: q_reg <= q_reg;
        endcase

    end
endmodule
```

PS: Resetting Your Counts to Zero

- **Reset**: A signal used to reset the register values to zero

- **Synchronous Reset**: Only takes effect when clock signal permits (for eg on posedge clk)
  - When we used `always @(posedge clk)` and `if(rst)`, we were using this pattern

- **Asynchronous Reset**: Takes effect whenever updated – independent of clock signal
  - More common in practice. Used as `always @(posedge clk or posedge rst)`

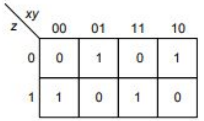
- The above is called an **"active-high" reset** which resets when the `rst` signal goes **high**
- In practice, **"active-low" reset** is more common, i.e., it resets when the `rst_n` signal goes **low**
- Use it as `always @(posedge clk or negedge rst_n)` -- we will use this more frequently

PS: Know your geography

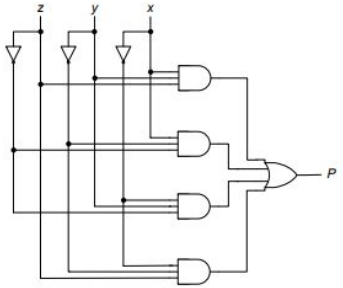
- Recall Karnaugh maps
- Why do we use them?
- What are don't-cares?
- Can you simplify boolean functions?

Combinational Circuits: Parity-bit Generator

Parity-bit generator: produces output value 1 if and only if an odd number of its inputs have value 1



(a) Map.



(b) Implementation.

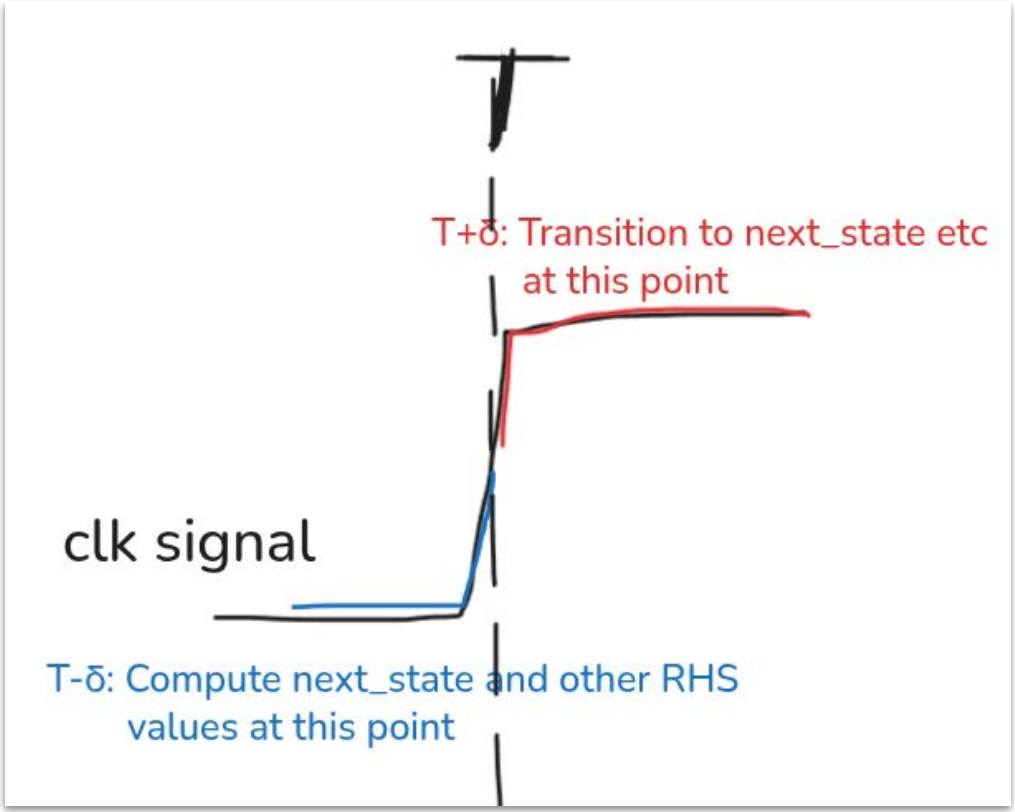
$$P = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z$$

- Practice?

1. Simplify the following Boolean function using a Karnaugh Map and express the final result in the minimal Sum of Products (SOP) form:

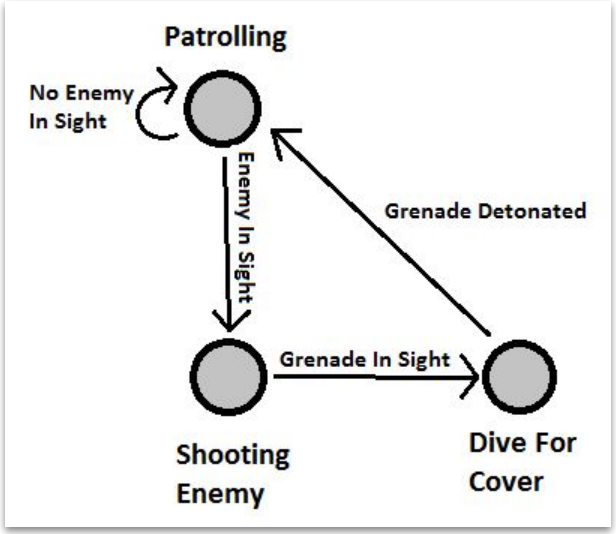
$$F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$$

PS: Clocked Sequential Circuits FTW

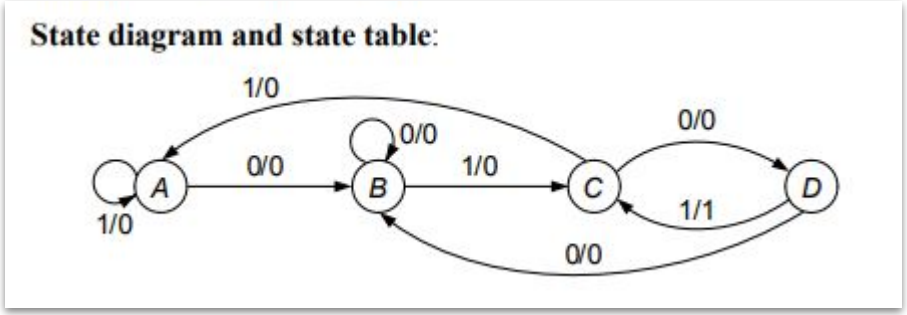


Why more Verilog?

- We will cover hardware design from a high level today!
- Also a common idiom in hardware design: finite state machines.
- **Finite state machines** define well-defined states (memory) and transitions (combinational logic) between them to consume some input and produce some output.
- Example: Bot AI in games be like



Eg: Simple 0101 detector



Plan for this lab

- Designing and writing a simple FSM-based circuit together
- First discuss, later you attempt as task1

- **Goals:**
  - Convert specifications to **high-level component design**
  - Write an **FSM** in Verilog: states, transitions, outputs, counters
  - **Combine smaller modules** to solve the entire problem



Traffic Control Simulator

P1. Bhavesh wants to go to his favourite restaurant right across JVLR road. But the traffic is horrible. Bhavesh is annoyed and blames the traffic light controller. He wishes to design a better traffic control system!

Goal: Build a chip that cycles through green->yellow->red phases with some fixed duration for each, and takes a pedestrian button as input – when pressed by a pedestrian, reduce the duration of green and increase that of red.

- **Q1**: What external inputs do we need? tick tock...
- **Q2:** What outputs do we need to produce?
- **Q3:** Do we need to store state for the circuit?
- **Q4**: What are the transitions, and when do they happen (ignore the pedestrian button for now)?
- **Q5:** Now add the pedestrian button. What should happen, precisely?
- **Q6:** Can we separate control from datapath? What goes where, and why?

Durations are as follows:

- *GREEN: Normally, **25** / If pedestrian request is on, **15***
- *RED: Normally, **30** / If pedestrian request is on, **40***
- *YELLOW: Always **5***

Traffic Control Simulator

- **Q1**: What external inputs do we need? tick tock...
  - Sequential/state-based circuit -> We need **clk, rst_n**
  - Button for pedestrian -> **ped_button**
  - Is clk a good way to keep track of time? Why/why not?
    - We need **tick_1hz**

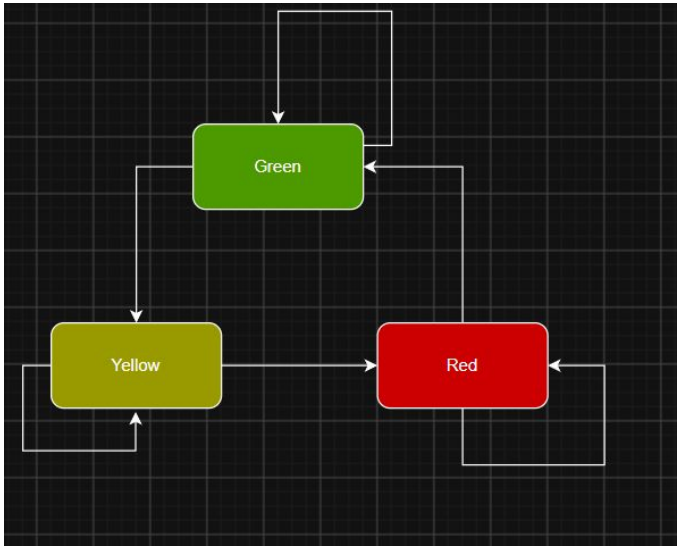
- **Q2:** What outputs do we need to produce?
  - One wire for each output: **red**, **yellow**, **green**

```
module bhavesh_traffic_light (
  input clk, rst_n,
  input ped_button,
  input tick_1hz,

  output red, green, yellow,
);
```

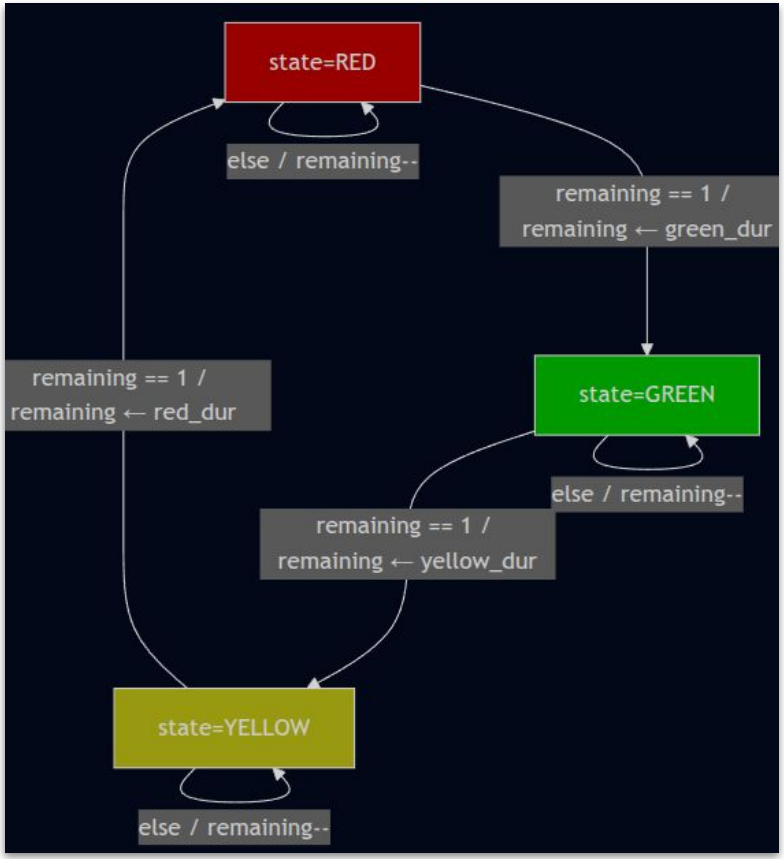
Traffic Control Simulator

- **Q3:** Do we need to store state for the circuit?
  - Yes. We need 3 states - red, yellow and green
  - Are we designing a **Moore** or **Mealy** machine?
    - Moore!
- **Q4:** What are the transitions, and when do they happen (ignore the pedestrian button for now)?
  - Green : Can stay at Green or go to Yellow
  - Yellow : Can stay at Yellow or go to Red
  - Red: Can stay at Red or go to Green



  - This is the **design**, how do we **implement** it?
  - Need a counter (**remaining**) that counts down for each state
  - We switch states when the counter is **at 1** (not 0) Why?

Abstraction!

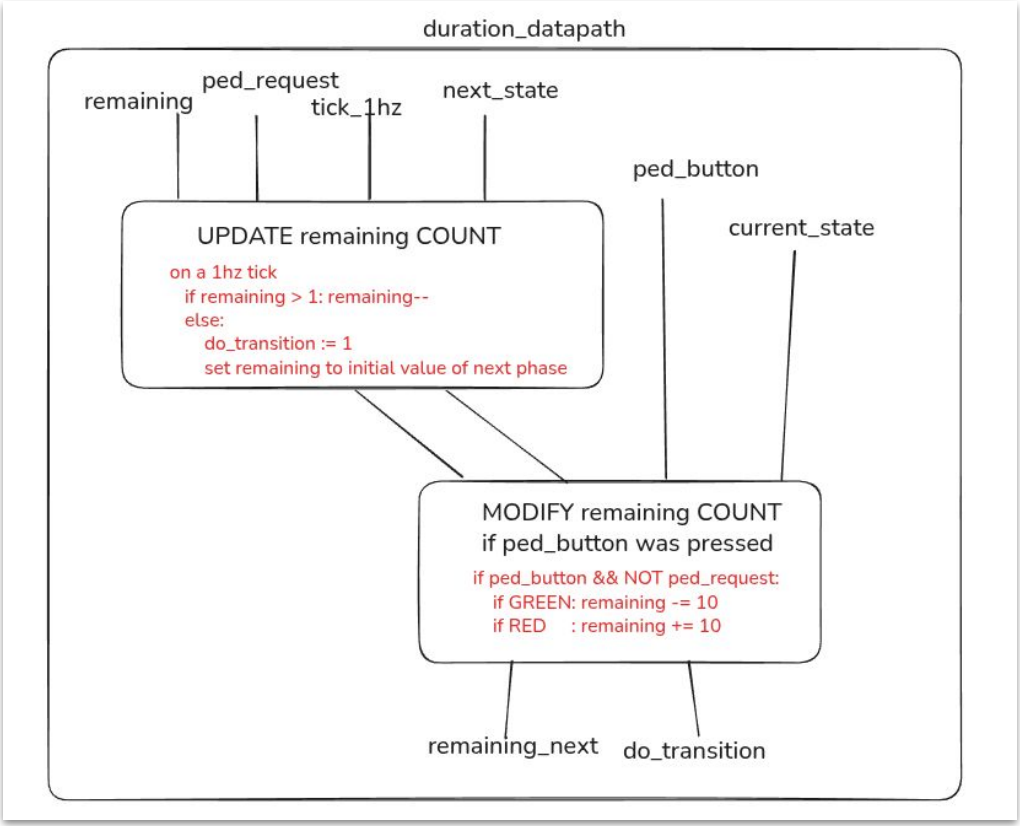


Traffic Control Simulator

Q5: Now add the pedestrian button. What should happen, precisely?

- Writing down intended behaviours on pressing button:
  a. Pressed during GREEN: GREEN durations shrinks (-10), RED extends (+10)
  b. Pressed during YELLOW or RED: RED extends (+10) because GREEN for this iteration is gone
- We need two behaviours:
  1. Instantly shrink/extend duration if the button press was during GREEN or RED phase
  2. "Remember" the press of a button until the end of this loop **(G->Y->R is a loop)**.
- For 1, when button is pressed, check if **state ∈ {GREEN, RED}** and do **-10/+10** to **remaining**
- For 2, latch the button until end of loop, and do a **+10** if transitioning **to RED** (from YELLOW)
  - Call the latch **ped_request**
- How to prevent long button press from triggering multiple -/+10s on GREEN?
  - Use **ped_button && !ped_request** for instantaneous triggering
- Did we miss something?
  - **Reset ped_request** after a transition out of RED

Traffic Control Simulator



Traffic Control Simulator

Q6: Can we separate control from datapath? What goes where, and why?

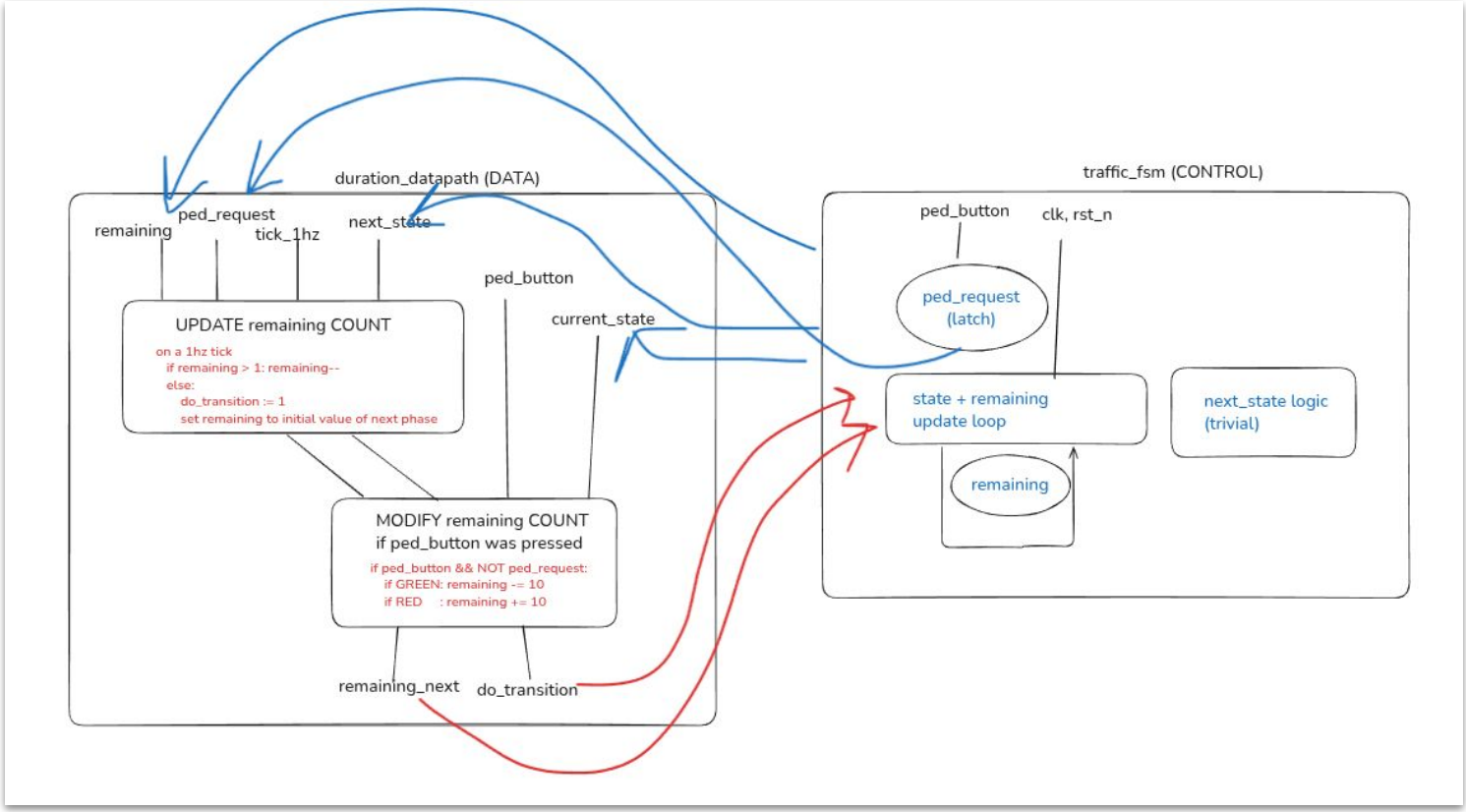
- What are the control signals here?
  - **tick_1hz**, **ped_button**, **ped_request**
- What is the “data” here?
  - **remaining** counter and **current_state**, **next_state**
- We separate the counter-tracking and updating logic entirely (datapath) into **duration_datapath**
  - Handles timer arithmetic, dealing with ped_button (fresh and old) – **combinational**
- We keep the control signals and state tracking inside **traffic_fsm**
  - Handles the overall state transitions based on reading/writing control signals – **sequential**

The two modules lived happily ever after?

Few extra questions and notes:

- We need a **do_transition** control signal to switch states... Who should produce this signal?
  - **duration_datapath!** Why?
- Do we need to store **next_state**?
  - Yes.
- Who should handle the latching of **ped_request** and computing **next_state**?
  - **traffic_fsm!**
- What should happen if someone presses ped_button on a “transition tick” (the final tick of a phase, at which we are going to switch states)? What issues can emerge?
  - Verdict: ignore these presses for now

Two modules happy ending



Plan for this lab

- Designing and writing a simple FSM-based circuit together

- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - Write an **FSM** in Verilog: states, transitions, outputs, counters
  - **Combine smaller modules** to solve the entire problem

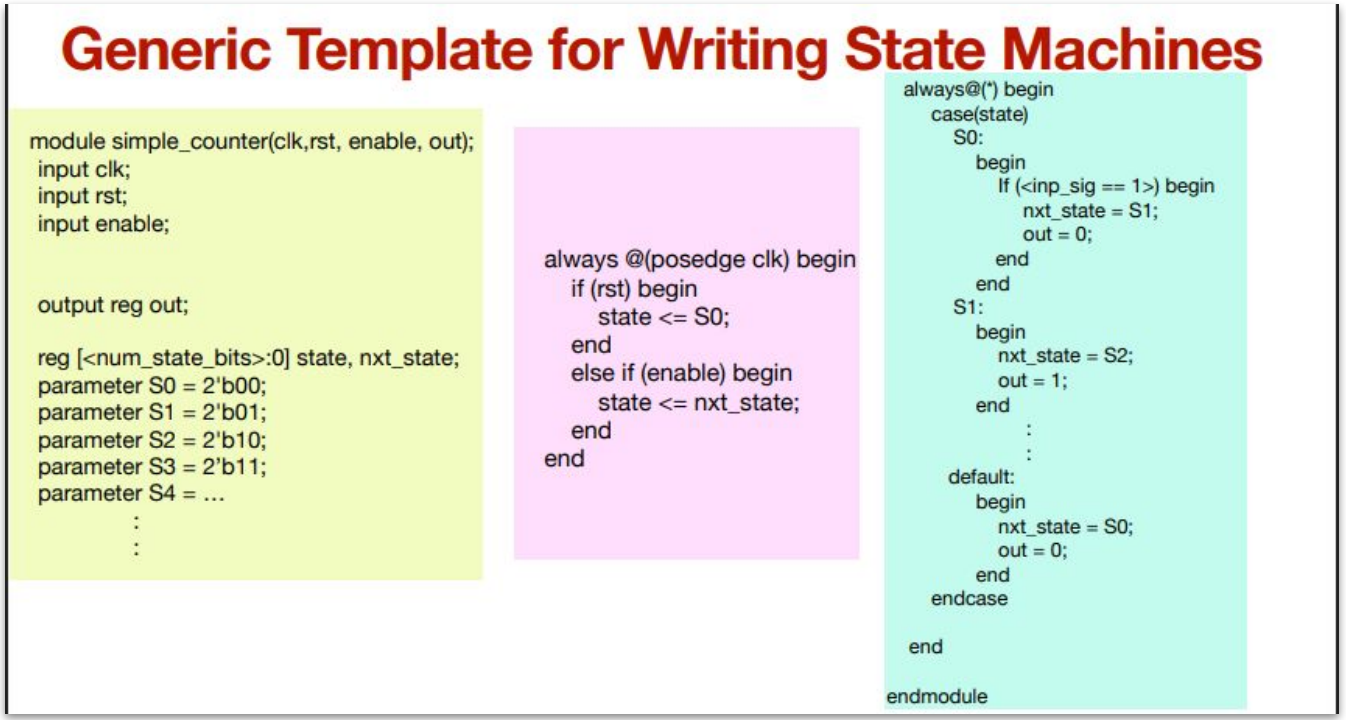


Recipe to a great FSM

1. Write one clocked, sequential `always` block to **model state updates**
2. Use `case` statement to **write the transitions/next_state for every state**
3. Use `<=` to apply changes to a **state register**

Recipe to a great FSM: Life is simple

- Recall this from CS230: don't read it in detail, get the high-level idea



Recipe to a great FSM

Do it on the board / VS Code!

Plan for this lab

- Designing and writing a simple FSM-based circuit together

- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - ~~Write an **FSM** in Verilog: states, transitions, outputs, counters~~
  - **Combine smaller modules** to solve the entire problem



Wiring is hard

- Have 3 components:
  1. **duration_datapath:** Inputs current **remaining** counter, curr/next **state**, **ped btn/req** and outputs updated **remaining_next** counter and **do_transition** signal
  2. **output_decoder:** Inputs **state** and outputs high on the wire corresponding to that state
  3. **traffic_fsm: (actual logic)** Inputs **clk, rst_n, ped_button, remaining_next** counter, **do_transition** signal and outputs **current_state, next_state, remaining** counter and **ped_request**

- Initialize all 3 and wire them up

- High-level module takes in inputs **clk, rst_n, tick_1hz, ped_button** and outputs **red/green/yellow**

Wiring it all up

```
module bhavesh_traffic_light (
    input clk, rst_n,
    input ped_button,
    input tick_1hz,

    output [1:0] state_out,
    output [7:0] duration_out,
    output red, green, yellow
);

    wire [1:0] current_state, next_state;
    wire [7:0] remaining, remaining_next;
    wire       ped_request;
    wire       do_transition;
```

```
    traffic_fsm fsm (
        .clk(clk),
        .rst_n(rst_n),
        .ped_button(ped_button),
        .next_state(next_state),
        .remaining_next(remaining_next),
        .do_transition(do_transition),
        .current_state(current_state),
        .remaining(remaining),
        .ped_request(ped_request)
    );

    duration_datapath dp (
        .current_state(current_state),
        .next_state(next_state),
        .remaining(remaining),
        .tick_1hz(tick_1hz),
        .ped_button(ped_button),
        .ped_request(ped_request),
        .remaining_next(remaining_next),
        .do_transition(do_transition)
    );

    output_decoder decoder (
        .state(current_state),
        .red(red),
        .green(green),
        .yellow(yellow)
    );
```

It’s inlab (1) time



Alice in TestbenchLand



Read testbench output and view waveforms:

- Reduce durations while testing to get a much shorter trace.
- Remove `clk` from the `$monitor` format string to suppress the lines where only the clock toggles
- Read the testbench code itself. Understand what each test is doing, then look for that behaviour in the monitor output.

Plan for this lab

- Designing and writing a simple FSM-based circuit together

- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - ~~Write an **FSM** in Verilog: states, transitions, outputs, counters~~
  - ~~**Combine smaller modules** to solve the entire problem~~



Plan for this lab

- Designing and writing a simple FSM-based circuit together



- **Goals:**
  - ~~Convert specifications to **high-level component design**~~
  - ~~Write an **FSM** in Verilog: states, transitions, outputs, counters~~
  - ~~**Combine smaller modules** to solve the entire problem~~
  - **genvar** and **parametric modules???**
    - **Optional (for outlab) and NOT in labexam**

Parametric is better

- A **`parameter`** is a constant you set at instantiation time. The module is written once, the width is filled in when you use it.
- Definition

```
module adder #(parameter N = 8) (
  input  [N-1:0] a, b,
  output [N-1:0] sum
);
  assign sum = a + b;
endmodule
```

- Usage

```
adder #(.N(8))  add8  (.a(a8),  .b(b8),  .sum(s8));
adder #(.N(16)) add16 (.a(a16), .b(b16), .sum(s16));
adder #(.N(32)) add32 (.a(a32), .b(b32), .sum(s32));
```

Looping over hardware

- Parameter allows you to specify variable width. But how do you write variable width hardware?
- Use **`generate`** and **`genvar`** to **generate units of hardware** in a **for loop**
- **`genvar`** is a loop variable that only exists at **elaboration time** (when the hardware is being built, before simulation runs)
- Eg: instantiate **N full_adder blocks** and wire them up

```
genvar i;
generate
  for (i = 0; i < N; i = i + 1) begin : bit_stage
    full_adder fa (
      .a(a[i]),
      .b(b[i]),
      .cin(carry[i]),
      .sum(sum[i]),
      .cout(carry[i+1])
    );
  end
endgenerate
```

Don’t keep looping over hardware!

- **Rule of thumb**: if you're wiring up **copies of a module** or **repeating assign statements**, use **`generate`**. If you're describing **what a circuit does** inside an always block, use a **regular for**.
- The latter is a behavioural description that runs at **simulation time**; the former generates hardware

```
// This creates 4 hardware AND gates
genvar i;
generate
  for (i = 0; i < 4; i = i + 1)
    assign out[i] = a[i] & b[i];
endgenerate

// This describes behaviour — runs every time inputs change
always @(*) begin
  for (i = 0; i < 4; i = i + 1)
    out[i] = a[i] & b[i];
end
```

Parameter is good, I like

- Parametric expressions can be used for indexing

```
wire feedback = crc_in[WIDTH-1];          // MSB, scales with WIDTH
wire [WIDTH-1:0] shifted = {crc_in[WIDTH-2:0], 1'b0};  // left shift by 1
```

- They can be used for filling out bits/replicating them. For eg: to initialize a register of size `WIDTH` to all ones: `parameter [WIDTH-1:0] INIT = {WIDTH{1'b1}};`
  - `{N{expr}}` means to repeat `expr`, N number of times and concatenate

- Remember **rule of thumb**: `generate/genvar` only help you when **instantiating parameterized hardware**, not when you're simply looping over indices, or using parametric indices etc.
- Above examples are proof that parametric DOES NOT IMPLY usage of generate.

It’s inlab (1) time



It’s graded lab time



1. Log in to:
   Username: **CS231-labexam**
   Password: …

2. Extract labexam tarball and work in the extracted folder

3. Go into the same folder and run
   `./build-archive.sh`

4. Log in to moodle and submit into the **correct activity: “Inlab Week 3 Assessment”**



LAB 4

x86-64 Assembly: Assemble!

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab

All previous labexam solutions are uploaded on the repo



Alert Slide

Please meet the TAs if you received **“Invalid Format” or similar comment** in a previous inlab



This you?

Why Assembly?

- **`1:1 Mapping`**: Assembly is the last step before raw machine code; one line ≈ one CPU instruction with a direct bijection.
- **`Zero Abstractions`**: There are no garbage collectors, heavy runtimes, or safety nets.
- **`Hardware Control`**: You directly control each and every register and byte.
- **`Maximum Speed`**: With proper assembly, you can beat compilers in execution speed.

With great power (of the CPU) comes great responsibility (for every single bit).

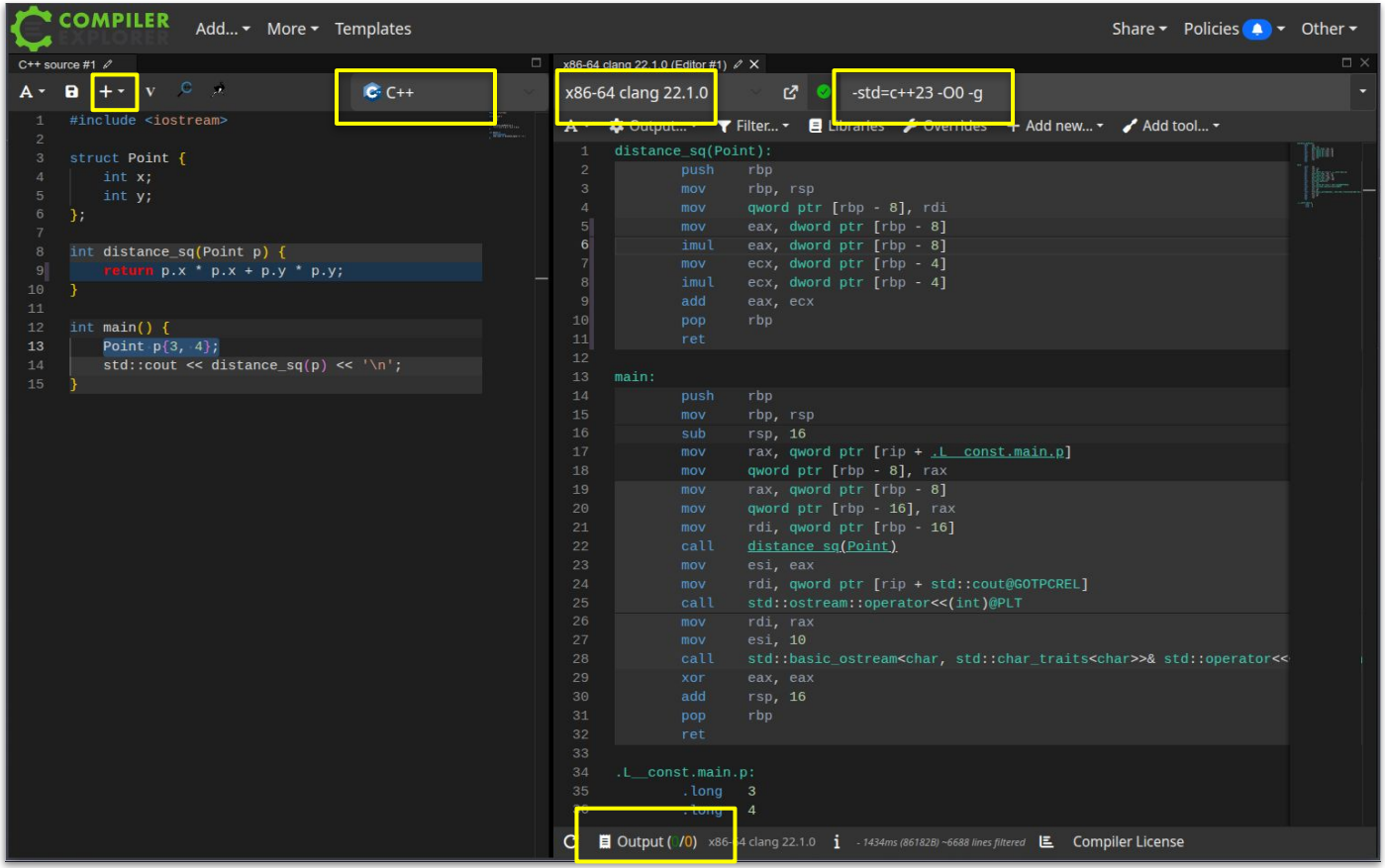
```
section .text
    global _start

_start:
    mov rax, 60 ; sys_exit
    xor rdi, rdi ; status: 0
    syscall

; Machine Code (12 bytes):
48 C7 C0 3C000000 48 31 FF 0F 05
```

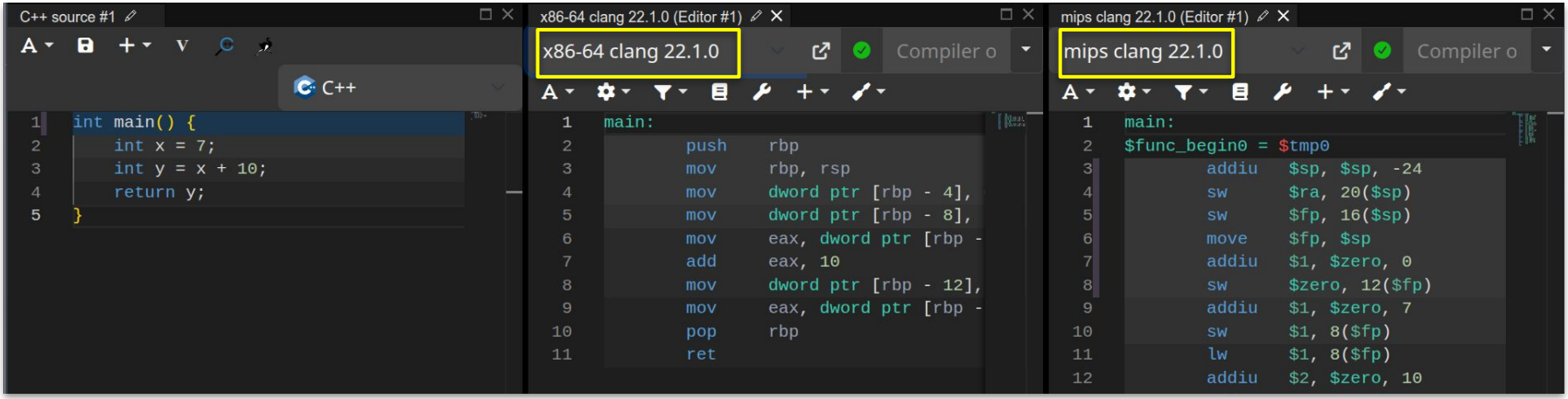
task0/exit.s

Godbolt it


COMPILER EXPLORER
Add...
More
Templates
Share
Policies
Other
C++ source #1
C++
x86-64 clang 22.1.0
-std=c++23 -O0 -g
Filter...
Add new...
Add tool...
#include <iostream>
struct Point {
int x;
int y;
};
int distance_sq(Point p) {
return p.x * p.x + p.y * p.y;
}
int main() {
Point p{3, 4};
std::cout << distance_sq(p) << '\n';
}
distance_sq(Point):
push rbp
mov rbp, rsp
mov qword ptr [rbp - 8], rdi
mov eax, dword ptr [rbp - 8]
imul eax, dword ptr [rbp - 8]
mov ecx, dword ptr [rbp - 4]
imul ecx, dword ptr [rbp - 4]
add eax, ecx
pop rbp
ret
main:
push rbp
mov rbp, rsp
sub rsp, 16
mov rax, qword ptr [rip + .L__const.main.p]
mov qword ptr [rbp - 8], rax
mov rax, qword ptr [rbp - 8]
mov qword ptr [rbp - 16], rax
mov rdi, qword ptr [rbp - 16]
call distance_sq(Point)
mov esi, eax
mov rdi, qword ptr [rip + std::cout@GOTPCREL]
call std::ostream::operator<<(int)@PLT
mov rdi, rax
mov esi, 10
call std::basic_ostream<char, std::char_traits<char>>& std::operator<<
xor eax, eax
add rsp, 16
pop rbp
ret
.L__const.main.p:
.long 3
.long 4
Output (0/0)
Compiler License

~~Compiler Explorer~~ Godbolt

- Use godbolt.org to see what your C/C++ code compiles to
- `clang` with `-O0 -g` gives nice readable code
- Click on **Output** at the bottom to see compiler stderr/stdout
- Click on some C/C++ code and press **`Ctrl+F10`** to see the corresponding assembly
- Click on the **+** button to add more tabs
- You can explore various compilers and see what the compiled output looks like



Sections in Assembly

Sections group data and instructions based on their access permissions, initialization requirements etc.

- **`section .data`** initialized storage: constants, arrays, buffers that exist before the program runs.
- **`section .text`** your actual instructions live here, organized under labels.
- **`global`** makes a symbol visible to the linker, allowing external object files or the runtime environment to reference it.
- **`_start`** is a special label marking the default entry point where program execution begins.

```
section .data
    a: dq 10
    b: dq 20

section .text
    global _start

_start:
    ; assemble here
```

task0/add.s

Labels in Assembly

- **`Labels`** act as named markers in your code, representing memory addresses of instructions or data.
- They end with a colon (**`:`**) when defined, serving as targets for jumps, loops, and function calls.
- **`jmp label`** redirects execution flow straight to the labeled instruction.

- **Recall from theory:** How do you track which instruction is being executed? What does a “jump” mean then?

- Can a label be used for a non-jump instruction?

```
section .text
global _start

_start:
; main execution point
    mov rcx, 5

.loop_start:
    dec rcx
    jnz .loop_start; jump if != 0
```

Registers: Big Potato Stuff

- **Registers** are tiny, blazing-fast storage slots built directly into the CPU taking almost zero time to access.
- **16 General-Purpose Registers**: rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp, r8–r15.
- The same GPR can be accessed in many bit-widths
- rax → eax → ax → (ah, al); rdi → edi → di → dil; r8 → r8d → r8w → r8b
- **Specialized Registers** also exist for floating-point/vector operations, status etc (such as **xmm0–xmm15** for 128-bit SIMD, **ymm0–ymm15** for 256-bit SIMD, **eflags**, the status register and many more).

One Register, Several Widths (Nesting of rax)



Basic Instructions

`mov dst, src`	copy a value into a **register** or **memory location**
`add/sub dst, value`	add / subtract, in place
`imul dst, value`	signed multiply
`xor reg, reg`	the idiomatic way to zero a register
`inc/dec dst`	increment / decrement by 1
`cmp a, b`	compute a − b, discard result, but update some flags
`push/pop value`	save / restore a value on the **stack**
`lea dst, addr`	compute a memory address without loading it



What is a value? Reg? Memory? Constant/Immediate value? What is addr? – **Coming Later**

Instruction Operands & Inputs

Assembly instructions perform operations on data, but where does that data come from?

The CPU handles three distinct types of **operands** (inputs and destinations) depending on where the values are physically stored and how quickly they need to be accessed.

Registers (reg)

Blazing-fast storage locations built directly inside the CPU core. Limited in number but near-instant to access.

```
mov rdi, rax
```

mov dst_reg, src_reg

Memory (*mem)

System RAM addresses, written inside brackets. Accessing memory is much slower than registers.

```
mov rax, [rbx]
```

mov reg, [mem_addr]

Immediates (imm)

Hardcoded constant values or raw numbers embedded directly within the instruction itself.

```
mov rbx, 42
```

mov reg, constant

Crucial Hardware Constraint: You **cannot** transfer data directly from memory to memory (e.g., '`mov [rsi], [rdi]`' is illegal).

Can you taste your code?

- Exact assembly syntax – order of arguments, what operands are allowed, some fancy symbols ($,%) – depends on the **syntax flavour**

- For eg: to move value in `rbx` to `rax` ⇒
  Intel: `mov rax, rbx`
  AT&T: `movq %rbx, %rax`

  - In AT&T, some instructions (like `mov`) have suffixes that tell the CPU what operand size to use or how to do bit extension
  - E.g. `b/w/l/q` specify size; `z` indicates zero-extension, as opposed to sign-extension



- How to view disassembled output from a compiled file?
  `objdump -d -M intel file.o` → To see Intel syntax
  `objdump -d -M att file.o` → To see AT&T syntax

- Or use godbolt.org :)

Assembly follows my brother

```c
static long a = 10;
static long b = 20;

long f(long a, long b) {
    return a + b;
}

int main() {
    return f(a, b);
}
```



```asm
section .data
    a: dq 10
    b: dq 20

section .text
    global _start

f:
    mov rax, rdi
    add rax, rsi
    ret

_start:
    mov rdi, [a]
    mov rsi, [b]
    call f

    mov rdi, rax
    mov rax, 60
    syscall
```

Mommy where's my data?

```
section .data
    a:      db 10           ; 1 byte
    b:      dw 1000         ; 2 bytes
    c:      dd 100000       ; 4 bytes
    d:      dq 10000000000  ; 8 bytes
    arr:    dq 1, 2, 3, 4   ; multiple values
    msg:    db "Hello", 0   ; string

section .bss
    buffer: resb 64         ; reserve 64 bytes
    arr2:   resq 10         ; reserve 10 × 8 bytes
```

- `.data` = **initialized data**
- `.bss` = **uninitialized storage**
- `db/dw/dd/dq` define 1/2/4/8-byte values, while `resb/resw/resd/resq` reserve 1/2/4/8-byte spaces
- label (eg: `a,b,arr,msg`) allows you to access these in code

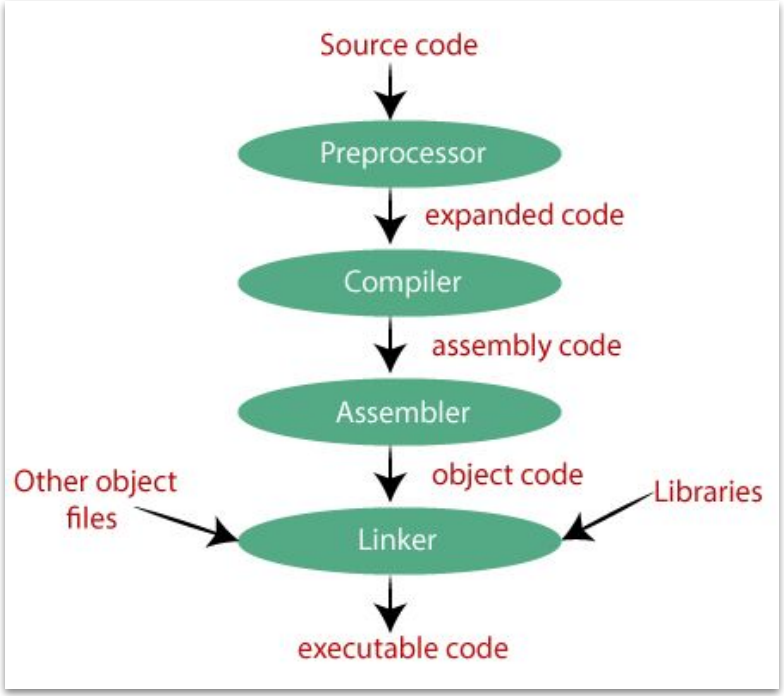
From Source Code to Executable

1. Source (.c) -> Assembly (.s)
   `$ gcc -S hello.c -o hello.s`

2. Assembly (.s) -> Obj File (.o)
   `$ nasm -f elf64 hello.s -o hello.o`
   -f specifies format (using ELF64 here)
   (Or, `objdump -d -M intel hello.o`)

3. Obj file (.o) -> Executable (.out or none)
   `$ ld hello.o main.o -o main`

- You can use **libc (standard C library)** (eg. printf etc.) in assembly by linking with it!
  - Easy: Use GCC: **`gcc hello.s -o hello`**
  - Or: Use ld: `ld hello.o -o hello -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2`



Avengers Assemble (tinyy potato) – tasko!



`add.s – move a into rdi, add b, exit with the result`

Jumps and Branching

- `cmp a, b` sets CPU flags based on (*a* − *b*). Subsequent jump instructions can use these flags to jump conditionally
- `jmp label` — unconditional, always taken.
- `je / jne` — jump if equal / not equal.
- `jl / jle / jg / jge` — signed < / ≤ / > / ≥.
- `jb / jbe / ja / jae` — unsigned < / ≤ / > / ≥.
- There's no if/else keyword — you build control flow yourself, one cmp and one jump at a time.
- An “if” in assembly is a comparison, a conditional jump past the body, and a label to land on.

```
jnc .else ; opposite of cond
.if:
; code to run if cond is true
jmp .end
.else:
; code to run when cond is false
.end:
```

task1 (medium potato): Rectangles!



`overlap.s` — do `rect_a` and `rect_b` overlap? 1 if yes, 0 if no

Arrays

- An array is just consecutive values in `memory`
- `dq 5, -2, 8, 3, -1, 6` defines six 64-bit values, back to back.
- **`[vec1 + 8*rcx]`** reads index `rcx`; we have to multiply `rcx` by `8` as the value is in bytes and each element is 8 bytes long here.
- Get the stride wrong and you silently read wrong bytes.
- Can 8 be replaced by any arbitrary integer?

```
vec1:
    dq 5, -2, 8, 3, -1, 6
.end:
vecsize equ (vec1.end - vec1)/8

    mov rsi, [vec1+8*rcx]
```

Memory Access Modes

[base + index*scale + displacement]

– In x86-64 assembly, the processor calculates address offsets in hardware using a single, unified instruction formula: **`[base + index*scale + displacement]`**.

– This powerful addressing mode lets you locate elements in arrays, structs, or matrices in a single instruction cycle without manual shift arithmetic.

Base `[reg1]`
Any 64-bit general-purpose register pointing to the start of the structure or memory block.

Index `[reg2]`
A register holding the dynamic index (e.g., loop counter) of the element to be accessed.

Scale `[* 2^x]`
A hardware multiplier: can only be `1, 2, 4,` or `8` to match standard element widths (`byte, word, dword, qword`).

Displacement `[+ C]`
A constant immediate value (offset) added to the address. Ideal for accessing a specific field within an array of structures or skipping headers.

- Eg: `mov eax, [rbx + rcx*4 + 8]`
- Equivalent to: `eax = *(int *)(rbx + rcx*4 + 8)`

Loops

- **No `for`/`while` keyword.** A loop in assembly is just four ingredients: a **label** to jump back to, a **bound check**, a **body**, and a **jump back to the check**. You assemble these yourself, every time.
- **The four-step pattern:**
  - **1. Init:** set up your index/accumulator with your starting values
  - **2. Check:** compare against bound (repeating condition)
  - **3. Body:** do the actual work for the iteration.
  - **4. Advance:** update the index, then jump back to re-check.
- **Where you check matters.** Both of the versions to the right do the same thing, but **removing the top check** in the upper version makes it a `do {} while()` loop

```
xor rcx, rcx ; i = 0
cmp rcx, N
jge .forend
.forbegin:
    ; ...
    ; loop body
    ; ...
    inc rcx
    cmp rcx, N
    jl .forbegin
.forend:
```

```
xor rcx, rcx ; i = 0
.forbegin:
    cmp rcx, N
    jge .forend
    ; ...
    ; loop body
    ; ...
    inc rcx
    jmp .forbegin
.forend:
```

Assemble task2: Dot Product!



`dotprod.s — loop, multiply, accumulate, exit with the result`

Who's the Impostor?

- Which for loop is correct/better to use?

```
.for:
    mov rsi, qword [vec1+8*rcx]
    imul rsi, qword [vec2+8*rcx]
    add rdi, rsi
    inc rcx
    cmp rcx, vecsize
    jl .for
```

```
.for:
    cmp rcx, vecsize
    jge .done

    mov rsi, [vec1 + 8*rcx]
    imul rsi, [vec2 + 8*rcx]
    add rdi, rsi

    inc rcx
    jmp .for
```

It’s Doomin’ Time

Food for thought:

1. x86 provides various operand combinations, as opposed to MIPS32 which mostly uses register–register operations. Do you think this makes x86 code faster, more convenient, both, or neither?

2. If registers are much faster to access than memory, why do CPUs use a relatively small number of registers instead of thousands of them?

3. `mov` is one of the most fundamental instructions in assembly. Do you think x86's mov alone is sufficient to express any computation?

4. (Explore) Are there x86 instructions that can do an arbitrary amount of “work”, as opposed to MIPS32 instructions which perform a fixed amount of work? What might be an example?

It’s graded lab time



Starting at 4:05.. Ending at ..

1. Log in to:
   Username: **CS231-labexam**
   Password: doomsday

2. Extract labexam tarball and work in the extracted folder

3. Go into the same folder and run
   `./build-archive.sh`

4. Log in to moodle and submit into the **correct activity: “Inlab Week 4 Assessment”**



Lab 5

x86-64 Assembly: Stack Overflow

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab

Outlabs 3 & 4 are out. Inlab 5 is as well.

All previous labexam solutions are uploaded on the repo



Plan for this lab

- ➔ How to handle **decimal** numbers and **Floating point** operations
- ➔ Breaking code into **functions** and understanding the **stack**
- ➔ **Syscalls :** Use some predefined functions which can be used to call the OS



XMM Registers

These are special 128 bit registers used for floating point arithmetic (wired up to the parts of the cpu which have capabilities of performing floating point arth)

- **`xmm0-xmm15`** are the 16 xmm registers present in 64 bit systems. (In 32 bit systems only 8 xmm registers are present)
- **`SSE Instruction`** are special instructions that are present in the ISA to perform operations with these registers.
- **`So how do we use them ?`**

```
add_cmplx:

    movsd xmm0, [rdi+8]
    movsd xmm1, [rdi+16]
    addsd xmm0, [rsi+8]
    addsd xmm1, [rdi+16]
    movsd [rdx+8], xmm0
    movsd [rdx+16], xmm1
    ret
```

Basic xmm/SSE instructions

`movsd xmm, xmm/*mem` `movsd xmm/*mem, xmm`	Can move data to / from memory location or other xmm
`addsd xmm, xmm/*mem` `subsd xmm, xmm/*mem`	add / subtract, in place
`mulsd xmm, xmm/*mem` `divsd xmm, xmm/*mem`	multiply / divide in place
`movq xmm, reg/*mem` `movq reg/*mem, xmm`	Can move data to / from general purpose register



We are only using **`64 bits`** out of the **`128 bits`** width in xmm registers

It’s inlab (1) time



Best Functions Forever <3

- **`call <label>:`** Pushes the address of the **next** instruction (also known as the return address) and jumps to the label.
- **`ret:`** Pops the return address from top of the stack and jumps back to respective address

```
main:
    call my_func
    ; execution resumes here

my_func:
    ; do work
    ret
```

Careful what you wish for

- What if we push and pop something else on the stack before doing ret?
- ret will read **something else** and it will not return :(



What is a stack?

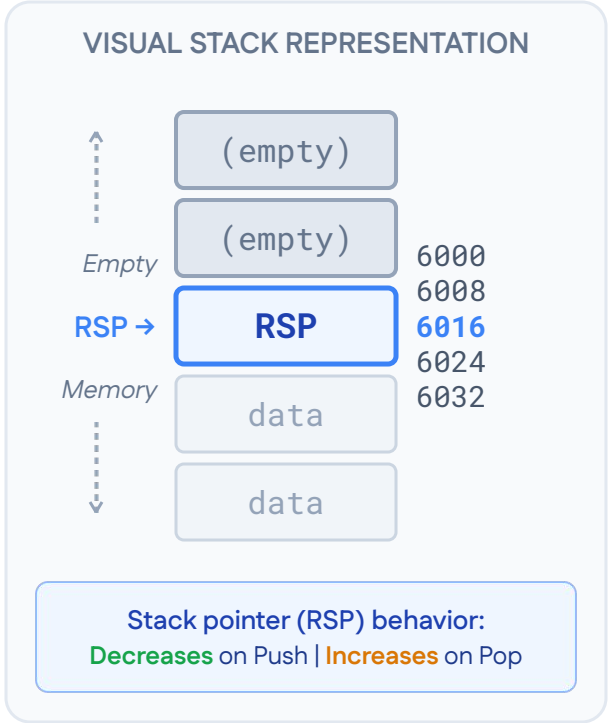
- Recall the function stack mumbo jumbo in c++
- We push things on the stack before a function call

  And pop them afterwards
- Stack grows downwards! (Higher -> lower memory addresses)
- Push operation: eg: `push rax`
- Equivalent assembly:

```
sub rsp, 8       ; say initially rsp=6016
mov [rsp], rax
```

- Pop operation: eg `pop rax`
- What should be equivalent assembly?

```
mov rax, [rsp]
add rsp, 8
```



Caller Callee Conventions

- **Caller saved registers:**
  - **Function arguments:** `rdi, rsi, rdx, rcx,r8, r9`
  - **Return value:** `rax`
  - **Scratch:** `r10, r11`
  - **All xmm registers:** `xmm0-xmm15`
- **Callee saved registers:**
  - **General Purpose Registers:** `rbx, rsp, rbp, r12, r13, r14, r15`



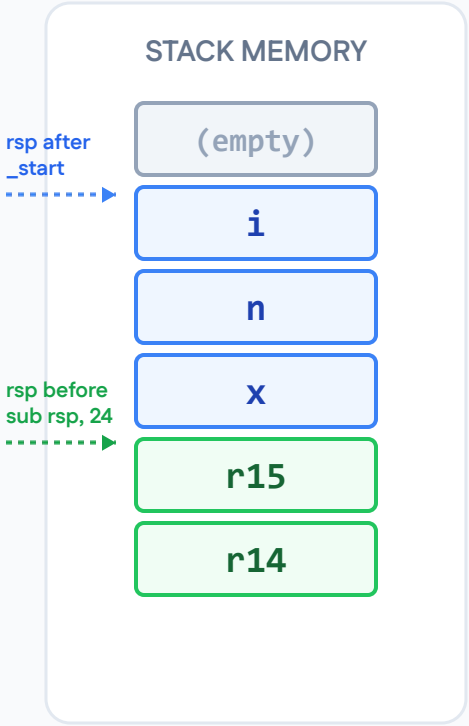
Example: Function

```
_start:
    push rbp
    push rsp
    push rbx
    push r12
    push r13
    push r14
    push r15

    sub rsp, 24

    mov rax, 0
    mov [rsp + 16], rax ; x = 0
    mov rax, 3
    mov [rsp + 8], rax ; n = 3

    mov rax, 0
    mov [rsp], rax ; i = 0
```



main.c

```
// Equivalent in C
long long x = 0;
long long n = 0;
long long i = 0;
```

Alignment is key

```
void example() {
    char a = 'A';  // 1 byte  (offset -1)
                   // 3 bytes of PADDING added here
    int b = 42;    // 4 bytes (offset -8, must be 4-byte aligned)
    char c = 'C';  // 1 byte  (offset -9)
                   // 7 bytes of PADDING added here
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)
}
```

- Recall memory alignment of data types from the theory course
- TLDR: **`sizeof(T) % alignof(T) == 0`**
- This is a standard practice (called SysV ABI) that C/C++ compilers implement for us
- In x86 we will have to manually insert “padding” to achieve this
- **Padding** – leaving some gaps in the memory address space

Alignment is key

```
void example() {
    char a = 'A';  // 1 byte  (offset -1)
                   // 3 bytes of PADDING added here
    int b = 42;    // 4 bytes (offset -8, must be 4-byte aligned)
    char c = 'C';  // 1 byte  (offset -9)
                   // 7 bytes of PADDING added here
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)
}
```

- For eg: for `int b` -> starting address should be ≡0 mod 4
- We insert padding suitable for this
- Remember xmm? Address should be 8 byte aligned while using `movsd` and `movaps/movapd` etc. Note: `movups` allows unaligned!

- When you call a function make sure its **16 byte aligned (at `call` boundary)**
- Especially when you use C functions like `printf` etc.. Why?

General functions and their convention

Register Arguments (Passed in Order)

1st Arg	2nd Arg	3rd Arg	4th Arg	5th Arg	6th Arg
`rdi`	`rsi`	`rdx`	`rcx`	`r8`	`r9`

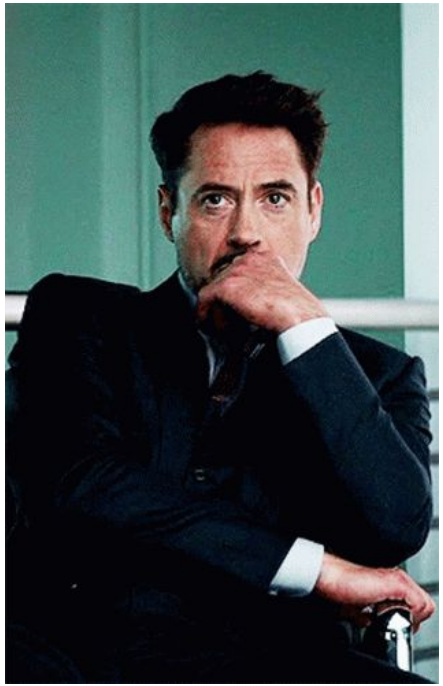
RETURN VALUE REGISTER

`rax`

Random questions

Common Doubts

- Why follow caller-callee convention?
- Why push/pop registers before & after every call?
- Why follow argument and return value conventions at all?



The Answer

MODULARITY

- Allows seamless integration with C function libraries.
- Enables using functions written by other people.
- Required to perform standard system calls.
- Mutual convention ensures code interoperability.

How do we use **C functions** in assembly?? (CHECK OUT OUTLAB)

Syscall me on my cell phone

What is a Syscall?

Requesting a privileged action from the OS (like writing to stdout).

The Mechanism

1. Load the syscall ID into `rax` (e.g., 1 for sys_write).
2. Load arguments using standard ABI rules (`rdi`, `rsi`, `rdx`).
3. Substitute `r10` in place of `rcx` if used
4. Execute the **`syscall`** instruction.

Position · Register · Description (sys_write Example)

Syscall Number `rax` (1 for `sys_write`)

1st Argument `rdi` (File Descriptor (`1` for `stdout`) for write)

2nd Argument `rsi` (Pointer to buffer string (`msg`) for write)

3rd Argument `rdx` (Length of buffer (`msglen`) for write)

**Note: Register arguments (rdi, rsi, rdx, etc.) hold entirely different meanings depending on the specific syscall invoked.*

It’s inlab (2 & 3) time



Caller callee convention done well

- Following are some slides to give a better understanding of caller callee convention
- Optional resources:
  - Calling convention (wiki)
  - A good outline of calling convention
  - (manual) the full sysv abi which goes way beyond just caller-callee register convention

- In short, registers are limited and common for any piece of code running on the CPU. This is a set of conventions for where and how to save each register.

Good old caller-callee

- A simple example of good old caller callee convention is on the repo here

CALLER – precall and postcall work

```
_start:
    ; GOAL: compute adder(10, 20) * 3 - 10

    mov rdi, 10 ; rdi - caller saved
    mov rbx, 3  ; rbx - callee saved

    ; PRE-CALL WORK : push caller-saved regs - rax,rdi,rsi,rdx,rcx,r8-11,xmm*
    ; Q1: Why am I not pushing rax? See below at NOTE1
    push rdi
    push rsi
    push rdx
    ; ...

    mov rdi, 10   ; put arguments in rdi, rsi...
    mov rsi, 20
    call adder    ; answer is put in rax

    ; POST-CALL WORK: pop caller-saved regs
    pop rdx
    pop rsi
    pop rdi
    ; recover old rdi, rsi, rdx values

    ; NOTE1: Does this usage give you the answer to Q1?
    imul rax, rbx ; rax *= rbx (rbx is untouched by `adder`, the callee)
    sub rax, rdi  ; rax -= rdi (rdi is pushed/popped by me , the caller)
    mov r12, rax  ; stash it away in r12, r12 is my returncode value

    ; answer (came in `rax`) should sit in `rdi`
    ; for setting returncode in syscall
    mov rdi, r12
    mov rax, 60
    syscall
```

CALLEE – prologue and epilogue

```
; receives arguments in rdi, rsi, rdx, rcx, ...
adder:
    ; PROLOGUE: push callee-saved regs - rbx,rsp,rbp,r12-15
    push rbx
    push r12

    ; do work!!
    ; here i can tamper with rbx
    mov rbx, rdi
    add rbx, rsi
    mov rax, rbx ; finally store return value in rax

    ; EPILOGUE: pop callee-saved regs - rbx,rsp,rbp,r12-15
    pop r12
    pop rbx

    ret
```

An Example with Godbolt

Simple C++ code and its assembly using the caller callee convention. [Godbolt link]

The code is on the repo as `sum.c` and `sum.s`. (Assembly is slightly modified for teaching)

- C++ code. Remember every variable is a register.
- Pause here and think which registers you should use for each variable – **caller-saved** or **callee-saved**. What are the tradeoffs?

```
1   float scale_add(float a, float b) {       // leaf
2       return a * 3.0f + b;
3   }
4
5   int sum_loop(float *arr, int n) {     // caller: has a value (total) live ACROSS the ca
6       float total = 0;
7       for (int i = 0; i < n; i++) {
8           total += scale_add(arr[i], i);   // total must survive this call
9       }
10      return total;
11  }
```

An Example with Godbolt

- Simple C++ code and its assembly using the caller callee convention. [Godbolt link]
- Callee (the leaf function code)
  - Do I really need to use `rbx`? Now imagine a world with callee-saved-only convention!

    Remember that this function is called in a **loop** multiple times

```
; ---- LEAF CALLEE: borrows rbx as scratch -> must save/restore it ----
scale_add(float, float):
        push    rbx             ; PROLOGUE: push rbx on stack
        mov     ebx, 3          ; ebx = 3   (just using rbx as scratch space)
        cvtsi2ss xmm2, ebx      ; xmm2 = 3.0
        mulss   xmm0, xmm2      ; xmm0 = a * 3.0
        addss   xmm0, xmm1      ; xmm0 += b     -> return value in xmm0
        pop     rbx             ; EPILOGUE: restore rbx
        ret
```

An Example with Godbolt

- Caller

```
; ---- CALLER ----
sum_loop(float*, int):
        xorps   xmm0, xmm0 ; total = 0.0, kept directly in xmm0
        mov     r12d, 0    ; i = 0
.loop_check:
        cmp     r12d, esi             ; i vs n (n is chilling in rsi)
        jge     .loop_end
        push    rdi                   ; PRE-CALL WORK: save 'arr'
        push    rsi                   ; PRE-CALL WORK: save 'n'
        push    r12                   ; PRE-CALL WORK: save 'i'
        sub     rsp, 8                ; PRE-CALL WORK: make room on stack for 'total'
        movss   dword ptr [rsp], xmm0 ; PRE-CALL WORK: save 'total'

        ; setup call
        movss   xmm0, dword ptr [rdi + 4*r12] ; xmm0 = arr[i]   (arg 1 for scale_add)
        cvtsi2ss xmm1, r12d                   ; xmm1 = (float)i (arg 2 for scale_add)
        call    scale_add(float, float)

        addss   xmm0, dword ptr [rsp] ; POST-CALL WORK: total += return value
        add     rsp, 8                ; POST-CALL WORK: free stack slot used for 'total'
        pop     r12                   ; POST-CALL WORK: restore 'i'
        pop     rsi                   ; POST-CALL WORK: restore 'n'
        pop     rdi                   ; POST-CALL WORK: restore 'arr'
        inc     r12d                  ; i++
        jmp     .loop_check
.loop_end:
        ret          ; total is already sitting in xmm0, nothing more to do
```